

计算机组成原理

期末复习·知识点重点整理（全7章）

适用考试日期：2026年6月12日 | 教材：计算机组成与设计（RISC-V版）

第1章 • 计算机系统概论

1.1 计算机发展历史

代别	年代	基础器件	运算速度	存储器	软件特征
第一代	1946-1957	电子管	几千~几万次/s	水银延迟线、磁鼓、纸带	机器语言
第二代	1958-1964	晶体管	几万~几十万次/s	磁芯、磁盘、磁带	FORTRAN、ALGOL-60、COBOL、汇编语言
第三代	1965-1970	集成电路 (IC)	几十万~几百万次/s	半导体、磁盘	操作系统、软件技术发展
第四代	1971-至今	超大规模集成电路 (VLSI)	MIPS → GIPS → TIPS	半导体、磁盘	微型计算机、外设迅速发展、多机处理/网络化

发展趋势：速度越来越快、体积越来越小、成本越来越低、功耗越来越低。

1.2 计算机体系结构中的7个伟大思想

7大思想

- 使用抽象简化设计**：通过分层抽象（如ISA、微架构）管理复杂度
- 加速经常性事件**：优化最常见的操作（如CPU加法、Cache命中路径）
- 通过并行提高性能**：指令级并行、数据级并行、多核并行
- 通过流水线提高性能**：将指令执行分成多个阶段，重叠执行
- 通过预测提高性能**：分支预测、数据预取
- 存储层次 (Memory Hierarchy)**：寄存器→Cache→主存→辅存，利用局部性
- 通过冗余提高可靠性**：ECC内存、RAID、备份

1.3 冯·诺依曼架构 vs 哈佛架构

特征	冯·诺依曼架构	哈佛架构
存储器结构	指令和数据共用同一存储器	指令存储器(IM)和数据存储器(DM)分开
总线结构	单总线（程序和数据共享）	双总线（程序和数据各自独立）
并行能力	取指和访存不能同时进行 → 结构冒险	取指和访存可同时进行
硬件复杂度	简单，成本低	复杂，需要两套地址+数据总线
典型应用	早期计算机（如X86外部总线）、嵌入式MCU	RISC-V流水线处理器、DSP、ARM
现代实现	现代处理器外部为冯·诺依曼，内部通过I-Cache/D-Cache实现类似哈佛架构（改进型哈佛）	

冯·诺依曼架构6大特征

- 计算机由**五大部件**组成（运算器、控制器、存储器、输入设备、输出设备）
- 指令和数据以**同等地位**存于存储器，可按地址寻访
- 指令和数据用**二进制**表示
- 指令由**操作码和地址码**组成
- 存储程序**：程序执行前装入存储器
- 以**运算器为中心**（指令通常顺序执行，特定条件下可改变顺序）

1.4 计算机硬件的主要性能指标

时间指标

- **主频 (f)**: CPU时钟频率, 单位 GHz
- **时钟周期 (T)**: $T = 1/f$, 单位 ns
- **CPI (Cycles Per Instruction)**: 平均每条指令所需时钟周期数
- **IPC (Instructions Per Cycle)**: $IPC = 1/CPI$
- **MIPS (Million Instructions Per Second)**: 百万指令/秒
- **MFLOPS**: 百万浮点运算/秒

核心公式 ★

- **CPU执行时间** = $IC \times CPI \times T = IC \times CPI / f$
- **IC** (Instruction Count) : 程序的总指令条数
- **MIPS** = 指令条数 / (执行时间 $\times 10^6$) = $f / (CPI \times 10^6)$
- **响应时间** (Response Time) : 从任务开始到结束的总时间
- **吞吐率** (Throughput) : 单位时间内完成的任务数

非时间指标

- **机器字长**: CPU一次处理的数据位数 (32位/64位)
- **总线宽度**: 数据总线的位数
- **主存容量**: 主存储器可寻址的空间大小
- **存储带宽**: 单位时间内存取的数据量
- **CPU内核数**: 多核处理器中的核心数量

第2章 • 计算机的运算方法

2.1 计算机中数的表示

2.1.1 无符号数与有符号数

基本概念

- **机器数**：数在计算机中的二进制表示形式（含符号位）
- **真值**：机器数对应的实际数值（带正负号的数值）
- **无符号数**：所有位均为数值位， n 位表示范围 $0 \sim 2^n - 1$
- **有符号数**：最高位为符号位(0正1负)，其余为数值位

四种机器数编码（8位字长，1位符号位）

- **原码**：符号位 + 绝对值。 $+0=0\ 0000000$, $-0=1\ 0000000$ 。
范围： $-(2^7-1) \sim +(2^7-1)$
- **反码**：正数=原码；负数=符号位不变，其余取反。
范围： $-(2^7-1) \sim +(2^7-1)$
- **补码**：正数=原码；负数=反码+1。只有1个0。
范围： $-2^7 \sim +(2^7-1)$ （多一个负数）
- **移码**：补码符号位取反。用于浮点数阶码。
便于比较大小：移码大的真值大

编码对比总结

- 正数：原码 = 补码 = 反码（符号位为0，数值位相同）
- +0和-0编码相同的是：**补码** 和 **移码**（即只有1个0）
- 补码 $\leftrightarrow$ 原码转换（负数）：①除符号位外每位取反+1；②自右向左从第一个1左边开始，每位取反（符号位除外）
- $[-X]_{\text{补}} = \text{将}[X]_{\text{补}}$ 连同符号位在内，每位取反，末位加1

2.1.2 定点表示与浮点表示

定点数

小数点位置固定：

	小数定点机	整数定点机
原码范围	$-(1-2^{-n}) \sim +(1-2^{-n})$	$-(2^n-1) \sim +(2^n-1)$
补码范围	$-1 \sim +(1-2^{-n})$	$-2^n \sim +(2^n-1)$
反码范围	$-(1-2^{-n}) \sim +(1-2^{-n})$	$-(2^n-1) \sim +(2^n-1)$

浮点数

$$N = S \times r^j$$

- **S**：尾数（小数，可正可负），**精度**
- **j**：阶码（整数，可正可负），**表示范围**
- **r**：基数（计算机中通常为2）
- 浮点数 = 阶符 + 阶码数值 + 数符 + 尾数数值
- **上溢**：阶码 > 最大阶码（真值超过表示范围）
- **下溢**：阶码 < 最小阶码（按机器零处理）

2.2 IEEE 754 浮点标准 ★

精度	总位数	符号位 s	指数位 E	尾数位 M	偏置值 Bias
单精度 (float)	32	1 bit [31]	8 bit [30:23]	23 bit [22:0]	127
双精度 (double)	64	1 bit [63]	11 bit [62:52]	52 bit [51:0]	1023

IEEE 754 编码规则

指数 E	尾数 M	表示
全0 (0)	= 0	± 0 （有+0和-0之分）
全0 (0)	$\neq 0$	非规格化数： $(-1)^s \times 0.M \times 2^{1-\text{Bias}}$
1 ~ 254 (单精度)	任意	规格化数 ： $(-1)^s \times 1.M \times 2^{E-\text{Bias}}$
全1 (255)	= 0	$\pm \infty$ （无穷大）
全1 (255)	$\neq 0$	NaN（非数）

关键点

- **规格化数有隐含的"1."**前缀：实际尾数值 = $1 + \text{尾数字段}/2^{23}$
- **指数 E 为无符号数**：实际指数 = $E - \text{Bias}$
- $E=0$ 和 $E=255$ 为特殊值，规格化数E范围：**1 ~ 254**（即实际指数 $-126 \sim +127$ ）
- 单精度最大规格化数： $\approx \pm 3.4 \times 10^{38}$
- 单精度最小正规格化数： $\approx 1.18 \times 10^{-38}$

例：将 -320 转换为 IEEE 754 单精度表示

$320 = 256 + 64 = 2^8 + 2^6 = 10100000_2 = 1.0100000 \times 2^8$
符号位 $s = 1$ (负数)
指数 $E = 8 + 127 = 135 = 10000111_2$
尾数 $M = 0100000000000000000000$ (23位, 去掉隐含的1)
结果: $1\ 10000111\ 01000000000000000000$
十六进制: $0xC3A00000$

2.3 算术移位与逻辑移位

移位类型	适用场景	移位规则	空位填补
算术移位	有符号数 (实现 $\times 2^n$ 或 $\div 2^n$)	符号位不变	—
算术左移	正数/负数	符号位不变, 数值位左移	低位补0 (正数原/补/反均补0; 负数原码补0, 补码/反码同右移规则)
算术右移	正数	符号位不变, 数值位右移	高位补0
算术右移	负数-原码	符号位不变, 数值位右移	高位补0
算术右移	负数-补码	符号位不变, 数值位右移	高位补1
算术右移	负数-反码	符号位不变, 数值位右移	高位补1
逻辑移位	无符号数	所有位一起移动	左移低位补0, 右移高位补0

例: 补码 10110010, 逻辑左移=01100100, 逻辑右移=01011001, 算术左移=11100100, 算术右移=11011001。
记忆: 补码负数算术右移补1 (保持符号), 其余情况补0。

2.4 定点运算

2.4.1 补码加减运算

补码加法公式

- 整数: $[A+B]_+ = [A]_+ + [B]_+ \pmod{2^{n+1}}$
- 小数: $[A+B]_+ = [A]_+ + [B]_+ \pmod{2}$
- 连同符号位一起相加, 符号位产生的进位自然丢掉

补码减法公式

- $A - B = A + (-B)$
- $[A-B]_+ = [A]_+ + [-B]_+$
- 求 $[-B]_+$ 的方法: $[B]_+$ 连同符号位取反 + 1

2.4.2 乘法运算

原码一位乘法

- 符号位单独处理: 乘积符号 = 被乘数符号 $\oplus$ 乘数符号
- 数值部分按绝对值相乘 (即无符号数乘法)
- 递推公式: $Z_0 = 0, Z_{i+1} = 2^{-1}(Y_{n-1}|X| \times 2^n + Z_i)$
- 用移位次数判断乘法是否结束
- 逻辑移位

Booth算法 (补码一位乘) ★

- 被乘数和乘数符号任意, 符号位自然形成
- 核心: 根据乘数末位 Y_0 和附加位 Y_{-1} 的差决定操作

Y_0	Y_{-1}	$Y_{-1} - Y_0$	操作
0	0	0	部分积右移1位
0	1	+1	部分积 $+[X]_+$, 再右移
1	0	-1	部分积 $+[X]_+$, 再右移
1	1	0	部分积右移1位

- 最后一步不移位

2.5 浮点四则运算

浮点加减运算步骤

① 对阶

- 求阶差 $\Delta j = j_x - j_y$
- 小阶向大阶看齐: 阶小的尾数右移1位, 阶码+1
- $\Delta j \neq 0$ 时, 阶小的尾数右移 $|\Delta j|$ 位 (可能丢失精度)

② 尾数求和/差

- 对阶后的两尾数按定点补码加减运算

③ 规格化

- **补码规格化判断**: 符号位与第1数位不同 (即 0.1xx... 或 1.0xx...)
- **左规**: 尾数不规格化时, 尾数左移1位, 阶码-1 (重复至规格化)
- **右规**: 尾数溢出 (双符号位01或10) 时, 尾数右移1位, 阶码+1
- 特例: $[-\frac{1}{2}]_+ = 1.100...0$ 按定义是规格化数, 但机器判为非规格化; $[-1]_+ = 1.000...0$ 按定义非规格化, 但机器判为规格化

④ 溢出判断

- 阶码 01, $\times \times \dots \times \rightarrow$ **上溢** (需中断处理)
- 阶码 10, $\times \times \dots \times \rightarrow$ **下溢** (按机器零处理)

2.6 汉明码 (Hamming Code) ★

汉明码三要素

1. **检测位数 k**: 满足 $2^k \geq n + k + 1$ (n为数据位数)
2. **检测位位置**: 放在 2^i 位置上 ($i=0,1,2,\dots$), 即第1,2,4,8,...位
3. **检测位取值**: 由所在检测小组的奇偶校验决定

检测小组分配 (Ci负责的位)

- **C₁** (位1): 位 1,3,5,7,9,11,... (二进制末尾为1的位, 即XXX1)
- **C₂** (位2): 位 2,3,6,7,10,11,... (二进制倒数第2位为1, 即XX1X)
- **C₄** (位4): 位 4,5,6,7,12,13,... (二进制倒数第3位为1, 即X1XX)
- **C₈** (位8): 位 8,9,10,11,12,13,14,15,24,... (X1XXX)

汉明码纠错

检测位 P_i 的形成 (以k=3为例)

- $P_1 = C_1 \oplus \text{位3} \oplus \text{位5} \oplus \text{位7}$
- $P_2 = C_2 \oplus \text{位3} \oplus \text{位6} \oplus \text{位7}$
- $P_4 = C_4 \oplus \text{位5} \oplus \text{位6} \oplus \text{位7}$
- 偶校验: 无错时 $P_1 = P_2 = P_4 = 0$
- $P_4 P_2 P_1$ 的二进制值 = 出错位的序号 (为0表示无错)

例: 数据 0101 按偶校验配置汉明码 (n=4, k=3): 位序 1 2 3 4 5 6 7 $\rightarrow C_1 C_2 0 C_4 1 0 1$ $C_1 = \text{位3} \oplus \text{位5} \oplus \text{位7} = 0 \oplus 1 \oplus 1 = 0$ $C_2 = \text{位3} \oplus \text{位6} \oplus \text{位7} = 0 \oplus 0 \oplus 1 = 1$ $C_4 = \text{位5} \oplus \text{位6} \oplus \text{位7} = 1 \oplus 0 \oplus 1 = 0 \therefore$ 汉明码 = **0100101**

ner">

第3章 • RISC-V汇编及其指令系统

3.1 RISC-V ISA 概述

RISC-V 核心设计理念

- RISC** 精简指令集：指令数量少、格式规整、寻址方式简单
- Load/Store 架构**：只有 **ld** / **sd** 等访存指令可以访问内存，所有运算操作数必须来自寄存器
- 寄存器-寄存器型**：ALU 运算的三个操作数（两源一目）全部在寄存器中
- 规整的指令格式：所有指令长度固定为 32 位（RV32），6 种格式共享相同的字段位置
- 模块化 ISA：基本整数指令集（RV32I / RV64I）+ 可选扩展（M/A/F/D/C 等）

32 个通用寄存器（RV64: 64位宽）

编号	ABI 名称	用途约定	保存者
x0	zero	恒为 0（写入被忽略）	—
x1	ra	返回地址 (Return Address)	调用者
x2	sp	栈指针 (Stack Pointer)	被调者
x3	gp	全局指针 (Global Pointer)	—
x4	tp	线程指针 (Thread Pointer)	—
x5-x7	t0-t2	临时寄存器	调用者
x8	s0/fp	保存寄存器 / 帧指针	被调者
x9	s1	保存寄存器	被调者
x10-x17	a0-a7	函数参数 / 返回值 (a0-a1)	调用者
x18-x27	s2-s11	保存寄存器	被调者
x28-x31	t3-t6	临时寄存器	调用者

记忆要点：被调者保存 (Callee-saved) = s0-s11 + sp + gp + tp；调用者保存 (Caller-saved) = ra + t0-t6 + a0-a7。

关键公式：叶子过程若使用的 s 寄存器不超过可用临时寄存器数量，可以完全不使用栈。

3.2 六种指令格式（细节展开）

格式	字段布局（位宽）	特征说明	代表指令
R-type	$\text{funct7}[31:25] + \text{rs2}[24:20] + \text{rs1}[19:15] + \text{funct3}[14:12] + \text{rd}[11:7] + \text{opcode}[6:0]$	三寄存器操作数， funct7+funct3 决定具体运算。 opcode=0110011	add, sub, sll, slt, sltu, xor, srl, sra, or, and
I-type	$\text{imm}[11:0][31:20] + \text{rs1}[19:15] + \text{funct3}[14:12] + \text{rd}[11:7] + \text{opcode}[6:0]$	含 12 位有符号立即数 （使用时会符号扩展至 32/64 位）。 移位指令 (slli/srli/srai) 中 funct7 的高 6 位用作额外立即数位，实际移位量 = imm[5:0]	addi, slti, xori, ori, andi, slli, srli, srai, ld, lw, lh, lb, jalr
S-type	$\text{imm}[11:5][31:25] + \text{rs2}[24:20] + \text{rs1}[19:15] + \text{funct3}[14:12] + \text{imm}[4:0][11:7] + \text{opcode}[6:0]$	立即数 拆成两段 存放 (imm[11:5] 在高位段，imm[4:0] 在低位段)，目的是让 rs1/rs2/rd 字段位置与其他格式对齐，简化译码	sd, sw, sh, sb
B-type	$\text{imm}[12 10:5][31:25] + \text{rs2}[24:20] + \text{rs1}[19:15] + \text{funct3}[14:12] + \text{imm}[4:1 11][11:7] + \text{opcode}[6:0]$	立即数 13 位有符号 (bit[0] 恒为 0，隐含 ×2)，可达 ±4KB 范围。 imm[12] 是符号位，imm[10:5] 和 imm[4:1 11] 拆分存放	beq, bne, blt, bge, bltu, bgeu
U-type	$\text{imm}[31:12][31:12] + \text{rd}[11:7] + \text{opcode}[6:0]$	仅含 20 位立即数 + 目标寄存器。无 rs1/rs2/funct3。 立即数占据指令的高 20 位，加载到 rd 的高 20 位（低 12 位补 0）	lui, auipc
J-type	$\text{imm}[20 10:1 11 19:12][31:12] + \text{rd}[11:7] + \text{opcode}[6:0]$	21 位有符号立即数 (bit[0] 恒为 0，×2)，跳转范围 ±1MB。 imm 字段被拆分得最碎：imm[20], imm[10:1], imm[11], imm[19:12] 四段	jal

为什么 S/B 型立即数要拆分？ 为了让 **rs1**、**rs2**、**rd**、**funct3** 在所有格式中**始终处于相同的 bit 位置**，从而极大简化译码电路的布线。代价是立即数重组需要额外的线序交叉。

各类型指令详解

R-type（寄存器-寄存器运算）

- add rd, rs1, rs2** — 加法
- sub rd, rs1, rs2** — 减法 (funct7=0x20)
- sll rd, rs1, rs2** — 逻辑左移
- srl rd, rs1, rs2** — 逻辑右移 (funct7=0x00)

I-type（立即数 & 访存 & 跳转）

- addi rd, rs1, imm** — 立即数加法
- slli/srli/srai rd, rs1, shamt** — 立即数移位 (shamt 只有 5/6 位有效)
- ld rd, offset(rs1)** — 加载双字 (64位)

- `sra rd, rs1, rs2` — 算术右移 (funct7=0x20)
- `slt rd, rs1, rs2` — 有符号比较, $rs1 < rs2 \rightarrow rd = 1$
- `sltu rd, rs1, rs2` — 无符号比较
- `and/or/xor rd, rs1, rs2` — 位运算

- `lw/lh/lb rd, offset(rs1)` — 加载字/半字/字节
- `lwu/lhu/lbu` — 无符号加载 (高位补0)
- `jalr rd, offset(rs1)` — 间接跳转: $rd = PC + 4, PC = (rs1 + offset) \& \sim 1$

S-type (存储指令)

- `sd rs2, offset(rs1)` — 存双字
- `sw/sh/sb rs2, offset(rs1)` — 存字/半字/字节
- $offset = imm[11:5] \parallel imm[4:0]$, 12 位有符号
- 地址 = $rs1 + sign_extend(offset)$

B-type (条件分支)

- `beq rs1, rs2, label` — 相等时分支
- `bne rs1, rs2, label` — 不等时分支
- `blt rs1, rs2, label` — 有符号小于
- `bge rs1, rs2, label` — 有符号大于等于
- `bltu/bgeu` — 无符号比较分支
- 目标地址 = $PC + offset \times 2$

U-type (上位立即数)

- `lui rd, imm20` — $rd = imm20 \ll 12$
用于构造 32 位立即数的高 20 位。例: `lui x5, 0x12345` $\rightarrow x5 = 0x12345000$
- `auipc rd, imm20` — $rd = PC + (imm20 \ll 12)$
用于 PC 相对寻址, 构建位置无关代码 (PIC)

J-type (无条件跳转)

- `jal rd, label` — $rd = PC + 4, PC = PC + offset \times 2$
- 若 $rd = x1(ra)$, 则为函数调用
- 若 $rd = x0(zero)$, 则为无条件跳转 (不保存返回地址)
- 伪指令 `j label` $\rightarrow$ `jal x0, label`
- 伪指令 `jalr x0, 0(ra)` $\rightarrow$ `ret`

3.3 寻址方式 (四种)

寻址方式	有效地址/操作数	对应指令类型	示例
立即数寻址	操作数 = 指令中的立即数 (符号扩展后)	I-type 运算类	<code>addi x5, x6, 100</code>
寄存器寻址	操作数 = 寄存器内容	R-type	<code>add x5, x6, x7</code>
基址+偏移 (变址)	$EA = rs1 + sign_extend(imm12)$	I-type(ld) / S-type(sd)	<code>ld x5, 40(x6)</code>
PC 相对寻址	$EA = PC + offset$ (B型: $\times 2$, J型: $\times 2$)	B-type / J-type	<code>beq x5, x6, loop</code>

3.4 过程调用 (Procedure Call) 完整流程

标准调用步骤 (6 步)

1. 传参: 将实参放入 `a0-a7` (超出 8 个的通过栈传递)
2. 跳转: `jal ra, func` (保存返回地址到 ra)
3. 开栈帧: `addi sp, sp, -N` (sp 向低地址增长, 分配 N 字节)
4. 保存现场: `sd sN, offset(sp)` 保存需保护的被调者寄存器
5. 执行过程体
6. 恢复现场+返回: `ld sN, offset(sp)` 恢复寄存器 $\rightarrow$ `addi sp, sp, N` 释放栈 $\rightarrow$ `jalr x0, 0(ra)` (ret)

叶子过程 (Leaf Procedure)

- 定义: 不再调用其他函数的过程
- 可以不使用栈 (如果只用 t0-t6 临时寄存器和 a0-a7 参数寄存器, 无需保护 ra, 因为 ra 未被修改)
- 如果要用的寄存器不够, 仍需使用栈保护 s 寄存器

3.5 汇编伪指令 & 常用模式

伪指令	等效真实指令	说明
<code>li rd, imm</code>	<code>lui + addi</code> (或单条 <code>addi</code>)	加载任意 32 位立即数
<code>mv rd, rs</code>	<code>addi rd, rs, 0</code>	寄存器间拷贝
<code>not rd, rs</code>	<code>xori rd, rs, -1</code>	按位取反
<code>neg rd, rs</code>	<code>sub rd, x0, rs</code>	取负数
<code>j label</code>	<code>jal x0, label</code>	无条件跳转
<code>ret</code>	<code>jalr x0, 0(ra)</code>	函数返回

la rd, symbol	auipc + addi	加载符号地址（位置无关）
nop	addi x0, x0, 0	空操作

3.6 案例分析要点

swap 过程

交换两个内存单元的值：

```
swap:
    slli t0, a1, 3      # 偏移 = k * 8
    add  t0, a0, t0      # 地址 = base + offset
    ld   t1, 0(t0)      # t1 = v[k]
    ld   t2, 8(t0)      # t2 = v[k+1]
    sd   t2, 0(t0)      # v[k] = t2
    sd   t1, 8(t0)      # v[k+1] = t1
    jalr x0, 0(ra)
```

sort 过程（嵌套循环）

外层循环（ $i: 0 \rightarrow n-1$ ），内层循环（ $j: i+1 \rightarrow n$ ），比较 $v[j]$ 和 $v[i]$ ，必要时调用 swap。关键：嵌套循环需要两个独立计数器，外层计数器需保存在 s 寄存器中（跨内层循环调用）。

数组遍历：指针 vs 索引

- 索引方式：每次循环需要 `slli + add + ld`（计算偏移+取数），循环体内指令更多
- 指针方式：直接用 `ld + addi`（指针递进），少一条 `slli` 指令
- 结论：指针方式更高效（每条迭代减少 1 条指令），编译器优化通常会索引转换为指针

栈帧布局

高地址→低地址：

- `fp(s0)` 指向帧顶部（保存的旧 `fp` + `ra`）
- 依次保存 s 寄存器
- 局部数组/变量（若需要）
- `sp` 指向帧底部（参数传递区，给被调函数用）

第4章 · 处理器设计

4.1 两大体系架构对比

特征	冯·诺依曼架构	哈佛架构
存储器	指令和数据共用同一存储器（单总线）	指令存储器 (IM) 和数据存储器 (DM) 分开（双总线）
并行性	取指和访存不能同时进行（结构冒险）	取指和访存可同时进行
硬件复杂度	简单，成本低	复杂，需要两套总线（地址+数据）
灵活性	指令和数据空间可动态分配	各自独立，空间固定
典型应用	早期计算机、嵌入式 MCU	RISC-V 流水线处理器、DSP
关键区别	单周期处理器使用冯·诺依曼（只有一个存储器）需注意取指与访存的冲突；流水线处理器一般采用哈佛架构（分离的 I-Cache 和 D-Cache 实现）	

4.2 单周期数据通路（完整组件图）

数据通路 = 组合逻辑（做运算）+ 状态单元（保存数据）。核心组件：

状态单元

- PC（程序计数器）：32/64 位寄存器，存当前指令地址
- 指令存储器 (IM)：按地址读取指令字
- 数据存储器 (DM)：按地址读写数据
- 寄存器堆 (Register File)：32×64 位，2 读端口 + 1 写端口

组合逻辑

- ALU：执行 AND/OR/ADD/SUB/SLT/NOR
- 加法器：PC+4、分支目标地址 (PC + offset×2)
- 立即数生成器 (ImmGen)：从指令字中提取并符号扩展立即数
- 多路选择器 (MUX)：ALUSrc、MemtoReg、PCSrc

寄存器堆写操作的特殊性：写操作在时钟上升沿触发（边沿触发写），读操作是组合逻辑（地址改变立即反映到输出）。因此可以在同一周期内先读后写（前一条写，后一条读同一寄存器 → 读到的是旧值，这正是流水线中需要前推的原因）。

4.3 ALU 设计（从 1-bit 到 32-bit）

1-bit ALU 核心结构

- AND 门：a & b
- OR 门：a | b
- 全加器 (Full Adder)：Sum = a⊕b⊕CarryIn, CarryOut = ab + (a⊕b)·CarryIn
- 4:1 MUX：选择输出结果 (AND/OR/ADD/其他)
- Binvert 信号：=1 时翻转 b，配合 CarryIn=1 实现减法 (a-b = a+~b+1)
- Ainvert 信号：=1 时翻转 a，用于 NOR (a NOR b = ~(a|b) = ~a & ~b)

32-bit ALU 组成与信号

- 由 32 个 1-bit ALU 级联 (CarryOuti → CarryIn_{i+1})
- ALU 控制线 (4bit)：

ALUOp	操作
0000	AND
0001	OR
0010	ADD
0110	SUB
0111	SLT
1100	NOR

- Zero 标志：所有 32 位输出 NOR 在一起 → 结果为 0 时输出 1（用于 beq）
- Overflow 检测：CarryIn[31] ⊕ CarryOut[31] = 1（最高位进位 ≠ 次高位向最高位的进位）

4.4 控制信号完整对照表

控制信号由指令的 opcode 字段通过 ROM（或组合逻辑）生成。ROM 的地址 = opcode (7bit)，输出 = 所有控制信号拼接。关键信号含义：

控制信号	位数	作用	=0 时	=1 时
------	----	----	------	------

ALUSrc	1	ALU 第二操作数来源	寄存器 (ReadData2)	立即数 (ImmGen 输出)
MemtoReg	1	写回寄存器的数据来源	ALU 结果	存储器读出数据
RegWrite	1	是否写寄存器堆	不写	写 (rd 地址)
MemRead	1	是否读数据存储器	不读	读
MemWrite	1	是否写数据存储器	不写	写 (上升沿)
Branch	1	是否为分支指令	非分支	分支 (beq等)
Jump	1	是否为跳转指令	非跳转	跳转 (jal)
ALUOp[1:0]	2	ALU 操作类型指示	00: ld/sd→加 01: beq→减	10: R型→由funct决定 11: —

典型指令的控制信号值

指令	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	Jump	ALUOp
R-type (add/sub...)	0	0	1	0	0	0	0	10
ld	1	1	1	1	0	0	0	00
sd	1	X	0	0	1	0	0	00
beq	0	X	0	0	0	1	0	01
jal	X	X	1	0	0	0	1	XX

4.5 ALU 控制逻辑

ALUOp[1:0]	funct3	funct7[5]	ALU Control[3:0]	操作
00	XXX	X	0010	ADD (用于 ld/sd 计算地址)
01	XXX	X	0110	SUB (用于 beq 比较)
10	000	0	0010	ADD
10	000	1	0110	SUB
10	111	0	0000	AND
10	110	0	0001	OR
10	010	0	0111	SLT

4.6 各指令的数据通路流程

R-type 数据流 (e.g. add x1, x2, x3)

PC→IM→[指令]→RegFile(读x2,x3)→ALU(运算,ALUSrc=0)→MUX(MemtoReg=0)→RegFile(写x1)
PC+4→PC (下一指令)

ld 数据流 (e.g. ld x1, 40(x2))

PC→IM→[指令]→RegFile(读x2)→ImmGen(符号扩展 40)→ALU(ADD,ALUSrc=1)→DM(读,MemRead=1)→MUX(MemtoReg=1)→RegFile(写 x1)
这是**最长的路径**，决定了时钟周期！

sd 数据流 (e.g. sd x1, 40(x2))

PC→IM→[指令]→RegFile(读 x1,x2)→ImmGen→ALU(ADD)→DM(写,MemWrite=1)
无写回寄存器操作，MemtoReg=无关

beq 数据流 (e.g. beq x1, x2, L)

PC→IM→[指令]→RegFile(读x1,x2)→ALU(SUB,若结果Zero=1则相等)
同时: PC+4 + (offset×2)→MUX(PCSrc=Zero-Branch)→PC

4.7 关键路径 (Critical Path) 分析

单周期处理器的时钟周期由 **ld** 指令的执行时间决定：

PC → **IM** (取指令, ~200ps) → **RegFile Read** (读寄存器, ~100ps) → **ALU** (计算地址, ~200ps) → **DM Read** (读数据, ~250ps) → **MUX** (MemtoReg, ~20ps) → **RegFile Setup** (写寄存器建立时间, ~50ps)

总延迟 ~ 所有阶段之和。其他指令 (R-type 等) 使用这条路径的子集，但时钟周期必须满足最慢指令。

ld 指令占有所有路径组件：IM + RegRead + ALU + DM + MUX + RegSetup，最完整的组合。

第5章 • 流水线处理器

5.1 五级流水线详解

阶段	简写	核心工作	产出
取指	IF	PC 送指令存储器，读出指令字；同时 PC+4 (加法器)	指令字、PC+4 → 存入 IF/ID
译码	ID	读 RegFile (rs1, rs2)；ImmGen 生成立即数；控制单元产生所有控制信号；检测数据冒险	ReadData1/2、Imm、控制信号、rd/rs1/rs2 地址 → 存入 ID/EX
执行	EX	ALU 运算 (数据运算或地址计算或分支比较)；分支目标地址计算；转发多路选择 (选择前推数据)	ALUResult、Zero、分支目标 → 存入 EX/MEM
访存	MEM	读/写数据存储器；分支决策 (若 beq 且 Zero=1, PC 更新为分支目标)	MemData、ALUResult → 存入 MEM/WB
写回	WB	选择写回数据 (MemtoReg MUX)，写入寄存器堆	完成指令

5.2 流水线寄存器（四组）详细内容

寄存器	位宽	存储内容
IF/ID	64+32	PC+4 (64b) + 指令字 (32b)
ID/EX	~300b	PC+4, ReadData1, ReadData2, sign_ext(Imm), funct3, funct7, rs1, rs2, rd, 控制信号: RegWrite, MemtoReg, Branch, MemRead, MemWrite, ALUSrc, ALUOp[1:0]
EX/MEM	~200b	PC+offset (分支目标), Zero, ALUResult, ReadData2 (写DM数据), rd, 控制信号: RegWrite, MemtoReg, Branch, MemRead, MemWrite
MEM/WB	~140b	ReadData (来自DM), ALUResult, rd, 控制信号: RegWrite, MemtoReg

流水线寄存器插入在相邻阶段之间，由同一时钟上升沿写入。每个时钟周期，所有流水线寄存器同时更新，数据向右“流动”一级。

5.3 流水线性能公式

基本公式

- 单周期: $T_{\text{single}} = \text{各级延迟之和}$
- 流水线: $T_{\text{pipe}} = \max(\text{各级延迟}) + T_{\text{reg}}$ (寄存器开销)
- 理想加速比 $= T_{\text{single}} / T_{\text{pipe}} \approx \text{流水线级数 } k$ (当各级均衡时)
- 实际加速比 $< k$, 原因: 流水线寄存器开销、各级不平衡、冒险停顿

CPI 详细计算

- 理想 CPI = 1 (每周期完成 1 条)
- 实际 CPI = 1 + stall_cycles_per_instruction
- stall 来源: Load-Use 停顿、分支预测错误惩罚、结构冒险等待
- 执行时间 = IC × CPI × T_{cycle}

5.4 流水线冒险 (Hazards) ★ 核心考点

5.4.1 结构冒险 (Structural Hazard)

产生原因

多条指令在同一时钟周期竞争同一硬件资源。
典型场景: 单端口存储器 — IF 阶段要读指令，同时 MEM 阶段要读/写数据 → 冲突。

解决方法

- 分离 IM/DM (哈佛架构)
- 分离 I-Cache / D-Cache (现代处理器的实现方式)
- 增加寄存器堆的读写端口 (RISC-V 已设计为 2 读 1 写)
- 插入 stall (性能损失大，不推荐)

5.4.2 数据冒险 (Data Hazard) ★★★

三种数据依赖类型

- RAW (Read After Write): 真依赖，后一条指令读前一条指令的写结果 → 在顺序流水线中必然发生
- WAW (Write After Write): 输出依赖，两条指令写同一寄存器 → 顺序流水线不会发生 (乱序执行才有)

检测与解决

- 在 ID 阶段进行冒险检测，比较当前指令的 rs1/rs2 与前面指令的 rd
- 检测条件 (以 EX 阶段的前一条为例):
`EX/MEM.RegWrite == 1`

- **WAR (Write After Read)**: 反依赖，后一条写，前一条读 → 顺序流水线不会发生（乱序执行才有）

EX/MEM.rd != x0

EX/MEM.rd == ID/EX.rs1（或 rs2）

转发路径 (Forwarding / Bypassing)

场景	前一条指令位置	数据来源	转发路径	需停顿?
EX 冒险 (1→2)	前一条在 EX，后一条在 ID	EX/MEM.ALUResult	EX/MEM → EX 的 ALU 输入 MUX	✗ 否
MEM 冒险 (1→3)	前一条在 MEM，后一条在 ID	MEM/WB.ALUResult	MEM/WB → EX 的 ALU 输入 MUX	✗ 否
Load-Use 冒险	前一条是 ld (在 EX)，后一条在 ID 要用该数据	MEM/WB.MemData（数据在 MEM 后才有效！）	MEM/WB → EX（但数据晚一个周期）	✅ 是！插入 1 个 stall (nop)
双数据冒险	两条前驱都可能提供数据	优先取 最近的前驱（EX/MEM 优先于 MEM/WB）	转发单元按优先级选择	✗ 否

Load-Use 冒险为什么必须停顿?

ld 指令在 **MEM 阶段结束时**数据才从 DM 读出。紧跟在后的指令在 EX 阶段就需要该数据（此时 ld 才刚到 MEM 阶段），数据尚未就绪。转发可以将 MEM/WB 的数据送到下一个周期的 EX，但中间差了一个周期，必须在 ID/EX 之间插入一个**流水线气泡 (pipeline bubble / nop)**，即：将 ID/EX 的控制信号清零（相当于插入一条 nop），让后续指令等待一个周期。

Hazard Detection Unit 的动作: 检测到 Load-Use → ①将 IF/ID 寄存器**保持**（同一指令重新译码）②将 PC 保持（不取新指令）③将 ID/EX 控制信号清零（插入气泡）

5.4.3 控制冒险 (Control Hazard) ★★

产生原因

分支/跳转指令**改变 PC**，但在流水线中分支结果（是否跳转、跳转目标）要到 **MEM 阶段 (beq)** 或 **EX 阶段末**才能确定。在此之前 IF 阶段已经取了两条错误路径的指令。

解决方法汇总

- **冲刷 (Flush)**: 分支确定后，将错误路径上已取的指令变为 nop（控制信号清零）。简单但每错一次浪费 2 个周期
- **提前分支决策**: 将分支比较和地址计算移到 ID 阶段（加比较器和加法器），只需 flush 1 条
- **预测不跳转**: 继续顺序取指，若实际跳转则 flush
- **静态预测**: 向后分支（循环）预测跳转，向前分支预测不跳转

分支预测器详解

预测器	状态数	原理	优缺点
1 位预测器	2 (跳转/不跳转)	记录上次该分支的执行结果。若上次跳了，这次预测跳	简单，但对交替分支模式（如循环末尾的交替 beq）每次都预测错
2 位预测器	4 (强不跳/弱不跳/弱跳/强跳)	饱和计数器: 跳转则 +1 (max 3)，不跳则 -1 (min 0)。预测方向由高位决定 (≥2 则预测跳)	需要连续两次误判才改变方向 ，对单次波动有容忍度
分支目标缓冲 (BTB)	—	Cache 结构: 用分支指令的 PC 索引，存储该分支的 目标地址 + 预测状态位。IF 阶段即查表，直接得出预测目标	减少分支目标计算延迟，但增加硬件开销

5.5 流水线中的异常与中断

关键寄存器

- **SEPC (Supervisor Exception PC)**: 保存异常/中断发生时的 PC 值，用于异常返回（执行 sret 时从 SEPC 恢复 PC）
- **SCAUSE (Supervisor Cause)**: 记录异常/中断的原因编码（如：2=非法指令, 5=Load 访问错误, 7=Store 访问错误）
- **STVAL**: 附加信息（如访问违例的地址）

流水线中异常处理原则

- **精确异常**: 异常指令**之前**的所有指令正常完成，异常指令**及其后**的所有指令被 flush
- 多级流水线中可能同时发生多个异常 → 取**最早阶段**的异常优先处理（IF 的指令地址错优先于 MEM 的数据访问错）
- 异常发生时，硬件自动: ①SEPC←异常指令PC ②SCAUSE←原因码 ③PC←异常处理入口地址 ④flush 后续指令

第6章 · 存储器

6.1 存储器分类（四个维度）

分类维度	类型	实例
存储介质	半导体存储器	RAM (SRAM/DRAM), ROM 系列
	磁表面存储器	硬盘 (HDD)、磁带
	光存储器	CD, DVD, Blu-ray
存取方式	随机存取 (RAM)	SRAM, DRAM — 任何地址存取时间相同
	顺序存取 (SAM)	磁带 — 必须按顺序通过前面的数据
	直接存取 (DAM)	磁盘 — 先定位到区域, 再顺序查找
在计算机中作用	高速缓存 (Cache)	SRAM, 位于 CPU 和主存之间
	主存储器 (Main Memory)	DRAM, 存放运行中的程序和数据
	辅助存储器 (Secondary)	SSD, HDD, 存放非活跃数据
	控制存储器	ROM, 存放 BIOS/固件/微程序
可保存性	易失性 (Volatile)	SRAM, DRAM — 断电丢失
	非易失性 (Non-volatile)	ROM, Flash, HDD — 断电保持

6.2 存储器层次结构 (Memory Hierarchy)

金字塔结构（自上而下）

层级	典型容量	访问时间	价格
寄存器	~1KB	~100ps	极贵
L1 Cache (SRAM)	~64KB	~1ns	贵
L2 Cache (SRAM)	~256KB-1MB	~5ns	
L3 Cache (SRAM)	~8-32MB	~15ns	
主存 (DRAM)	~8-64GB	~50ns	中等
SSD (Flash)	~256GB-2TB	~100μs	便宜
HDD (磁盘)	~1-10TB	~5ms	极便宜

局部性原理 (Locality) ★

时间局部性 (Temporal Locality):

- 刚被访问过的数据项短期内很可能再次被访问
- Cache 保存最近访问的数据

空间局部性 (Spatial Locality):

- 某地址附近的数据项短期内很可能被访问
- Cache 以块 (Block) 为单位从主存加载

两种局部性是 **Cache 有效的理论基础**。程序中的循环、顺序访问、栈操作等都表现出强局部性。

6.3 半导体存储器对比

特性	SRAM	DRAM
存储单元	6 个晶体管 (双稳态触发器)	1 个晶体管 + 1 个电容
集成度	低 (单元面积大)	高 (约为 SRAM 的 4-6 倍)
速度	快 (~1-10ns)	慢 (~50-70ns)
功耗	静态功耗较高 (持续电流)	静态功耗低, 但需刷新功耗
是否需要刷新	否 (双稳态自保持)	是 (电容漏电, 通常每 64ms 刷新一遍)
破坏性读出	否	是 (读取后需写回)
典型用途	Cache (L1/L2/L3)	主存储器 (Main Memory)
相对价格	高	低 (约为 SRAM 的 1/10-1/20)

ROM 系列对比

类型	全称	编程方式	擦除方式	可重写次数
MROM	Mask ROM	工厂掩膜固化	不可擦除	0 (一次成型)
PROM	Programmable ROM	用户一次编程 (熔丝烧断)	不可擦除	1 次
EPROM	Erasable PROM	电编程	紫外线照射 (20-30 分钟)	约 100-1000 次

EEPROM	Electrically EPROM	电编程	电擦除 (可字节级)	约 10^4 - 10^6 次
Flash	Flash Memory	电编程	电擦除 (块级, 比 EEPROM 快)	约 10^5 次

6.4 DRAM 刷新

为什么需要刷新

DRAM 用电容存储信息。电容存在漏电流, 电荷逐渐流失, 信息在数 ms 至数十 ms 内丢失。必须定期重新充电 (读出后放大再写回)。

三种刷新方式

- 集中式**: 在一段连续时间内刷新所有行 (简单, 但有"死区")
- 分散式**: 将刷新操作分散到每个存储周期后 (无死区, 但频繁打断)
- 异步式**: 在 CPU 不访问存储器时刷新 (效率最高, 但控制复杂)

6.5 Cache — 核心知识点

6.5.1 地址映射 (三种方式)

映射方式	规则	命中率	比较器开销	冲突缺失
直接映射	主存块 j 只能映射到 Cache 行 $(j \bmod N)$, N = Cache 行数	最低	1 个比较器	最多 (多块竞争同一行)
全相联映射	主存块可映射到任意 Cache 行	最高	N 个比较器 (全并行比较)	无冲突 (只有容量缺失)
k 路组相联	主存块 j 映射到组 $(j \bmod S)$, 组内任选 k 个行之一	折中	k 个比较器	折中

6.5.2 地址结构

地址划分

主存地址 = **Tag** (标记) + **Index** (索引) + **Block Offset** (块内偏移)

- Block Offset 位数 = $\log_2(\text{块大小, 字节})$
- Index 位数 = $\log_2(\text{Cache 组数})$ 【直接映射: 组数=行数; 全相联: 组数=1】
- Tag 位数 = 地址总位数 - Index 位数 - Offset 位数

典型计算示例

主存 4GB (32 位地址), Cache 64KB, 块 64B, 4 路组相联:

- Offset = $\log_2 64 = 6$ 位
- Cache 总行数 = $64\text{KB} / 64\text{B} = 1024$ 行
- 组数 = $1024 / 4 = 256$ 组
- Index = $\log_2 256 = 8$ 位
- Tag = $32 - 8 - 6 = 18$ 位

6.5.3 替换算法 & 写策略

替换算法 (四种)

- RAND**: 随机, 实现简单, 命中率不稳定
- FIFO**: 替换最早进入的块, 不反映局部性
- LRU**: 替换最久未使用的块, **命中率最高**, 但需要跟踪使用历史 (硬件开销大)
- 伪 LRU (Pseudo-LRU)**: 近似 LRU, 常用二叉树算法, 开销低, 实际中广泛使用

写策略

- 写直达 (Write-through)**: 同时写 Cache 和主存。一致性易保证, 但写开销大 (每次写都要访问主存), 可通过写缓冲 (Write Buffer) 缓解
- 写回 (Write-back)**: 仅写 Cache, 该块被替换出时才写回主存。写速度快, 但需要 dirty 位标记, 一致性维护复杂
- 写分配 (Write-allocate)**: 写缺失时, 先将该块从主存加载到 Cache, 再写 Cache (通常配合 Write-back)
- 非写分配 (No-write-allocate)**: 写缺失时直接写主存, 不加载到 Cache (通常配合 Write-through)

6.6 辅助存储器

类型	原理	容量	速度	特点
SSD	NAND Flash 芯片阵列	256GB-4TB	读 ~100μs, 写 ~1ms	无机械部件, 抗震, 静音, 价格逐年降低
HDD	磁介质盘片+磁头臂	1TB-20TB	寻道 ~5-10ms	容量大、极便宜, 但有机械延迟和故障风险

第7章 • 系统总线与输入输出

7.1 系统总线

总线分类

- 按功能：地址总线（单向，CPU→设备，宽度决定寻址范围）、数据总线（双向，宽度影响吞吐率）、控制总线（传输握手/时序/命令信号）
- 按位置：片内总线、系统总线（板级）、通信总线（外部/网络）
- 按结构：单总线（简单但带宽瓶颈）、双总线（CPU-主存 + I/O 分开）、多总线（分层/桥接，现代计算机标准）

总线特性与带宽

- 总线宽度：数据线的条数（如 32 位、64 位）
- 工作频率：每秒传输的次数（Hz）
- 带宽公式：带宽 = 总线宽度 × 工作频率
（单位：B/s 或 bps；若一个时钟传多次数据，需乘倍数）
- 例：64 位总线 @ 100MHz → 64b×100M = 6400Mbps = 800 MB/s

7.2 总线仲裁

仲裁方式	原理	优先级	可靠性	硬件开销
菊花链 (Daisy Chain)	总线许可信号沿链式线路从近到远传递，最近的设备优先获得	固定（物理位置决定，近高远低）	低（单点故障影响下游）	最小（3条控制线）
集中式并行仲裁	每个设备有独立的总线请求线和许可线，集中仲裁器决定	可编程（灵活）	中（仲裁器单点故障）	大（每个设备一条线）
分布式仲裁	各设备自行判断（通过比较请求线上的 ID），无须集中仲裁器	可编程	高（无单点故障）	中

7.3 总线通信方式

方式	时钟	握手	优点	缺点	适用场景
同步通信	统一时钟	无	简单、快速、时序确定	所有设备必须同频，无法兼容慢速设备	CPU-主存
异步通信	各自时钟	请求/应答握手	兼容不同速度设备，灵活	握手开销大，每次传输握手约增加 2-3 个时钟周期	跨时钟域 I/O
半同步通信	统一时钟 + 等待	就绪信号	折中：快设备同步，慢设备可插入等待周期	增加等待信号的判断逻辑	混合速度外设总线
分离式通信	各自时钟	有	请求和响应分两次总线事务，期间总线可给其他设备用。总线利用率最高	控制最复杂，需要事务 ID 匹配	高性能总线（如 AXI）

7.4 I/O 控制方式 ★ 重点对比

特征	程序查询（轮询）	程序中断	DMA
数据传送单位	字 / 字节	字 / 字节	数据块（多字节）
启动方式	CPU 主动循环检查状态寄存器	外设主动发中断请求	CPU 初始化 DMA 控制器参数后启动
CPU 介入程度	全程参与（100% CPU 时间用于轮询）	仅在中断响应时介入传送和恢复现场	仅在开始（初始化）和结束（中断通知）时介入
并行性	无（CPU 与外设串行工作）	CPU 与外设部分并行（传送期间 CPU 可执行其他程序）	高度并行（CPU 和 DMA 同时工作，交替使用总线）
硬件开销	最小（仅状态寄存器）	中等（中断控制器）	大（DMA 控制器、总线仲裁）
适用场景	简单系统、已知等待时间的外设	低速/中速、交互式 I/O（键盘、鼠标）	高速大批量传输（磁盘、SSD、网卡、显卡）

7.5 中断系统

中断处理完整流程

- 外设发 中断请求
- CPU 执行完当前指令
- 关中断（防止嵌套干扰）
- 保存断点（PC → 栈 或 SEPC）

中断优先级 & 嵌套 & 屏蔽

- 优先级判断方式：软件查询（按查询顺序，灵活但慢）、硬件链式（菊花链，固定优先级）、中断控制器（如 RISC-V PLIC，可编程）
- 中断嵌套：高优先级中断可打断低优先级 ISR。条件：①中断允许标志置位 ②新中断优先级更高 ③当前 ISR 至少已保存了关键上下

5. 识别**中断源**（软件查询 / 硬件向量 / 中断控制器）
6. 转入 **中断服务程序 (ISR)**
7. 执行 ISR → **恢复现场**
8. **开中断** → **返回**（sret / iret）

- 文
- **中断屏蔽**：通过屏蔽寄存器控制各中断源的允许/禁止。CPU 可通过设置屏蔽字实现选择性响应

7.6 DMA 详解

三种 DMA 工作方式

方式	传送方式	总线利用率	CPU 影响
突发 (Burst)	连续传送整个块	DMA 独占期间 100%	CPU 暂停 (无法访存)
周期窃取 (Cycle Stealing)	每次传 1 字, 释放总线	交替使用	CPU 略微减速
透明 (Transparent)	仅在 CPU 不用总线时传	最高	无影响 (完全透明)

DMA 工作过程

1. **初始化**：CPU 向 DMA 控制器写入：源地址、目的地址、传送字节数、方向（读/写）、启动位

2. **总线请求**：DMA 控制器向 CPU 发出 HRQ (Hold Request)

3. **总线授权**：CPU 完成当前总线周期后释放总线，发出 HLDA (Hold Acknowledge)

4. **数据传送**：DMA 接管总线，产生地址和读写控制信号

5. **传送完成**：字节计数归零 → DMA 释放总线 → 向 CPU 发中断（EOP 信号）

DMA 与中断 + 程序查询的对比总结

比较维度	程序查询	程序中断	DMA
CPU 利用率	极低（全程等待）	中（仅在传送时占用）	极高（基本不占用）
数据传输率	低（受 CPU 指令速度限制）	中（每次中断开销几十~几百周期）	高（接近总线带宽）
响应速度	慢（轮询周期限制）	快（硬件中断响应）	最快（硬件直接传送）
典型数据速率	< 1 KB/s	1-100 KB/s	> 1 MB/s 至 GB/s

重点知识点填空练习（共80空，满分100分）

说明：请在横线处填入正确的术语、数值或陈述。每空独立评分，覆盖第1-7章全部核心概念。下划线长度不暗示答案长度。

第1章 · 计算机系统概论（10题）

- 冯·诺依曼架构中，计算机由五大部件组成：运算器、_____、_____、输入设备和输出设备。
- 冯·诺依曼架构的核心思想是_____，即程序执行前必须装入存储器。
- 第一代计算机使用_____作为基础器件，第四代使用_____。
- 计算机体系结构7个伟大思想中，“通过_____提高性能”是流水线处理器设计的基础。
- CPU执行时间 = _____ × CPI × _____。
- MIPS（百万指令每秒）= _____。（用主频和CPI表示）
- 哈佛架构与冯·诺依曼架构的主要区别在于_____和_____是否分离。
- 计算机层次结构中，位于硬件和软件之间的分界面是_____（英文缩写）。
- 系列机的核心特征是基本_____相同、基本_____相同，以保证软件兼容。
- 现代处理器外部总线为冯·诺依曼结构，但由于内部_____的存在，实际上类似于_____。

第2章 · 计算机的运算方法（18题）

- 机器数的四种编码方式为：原码、_____、_____和_____。
- 在四种编码中，+0和-0编码相同（即只有一个0）的是_____和_____。
- 8位补码表示整数的范围是_____到_____。
- 已知 $[X]_2$ ，求 $[-X]_2$ 的方法是：_____。
- IEEE 754单精度浮点数共_____位，其中符号位_____位、指数位_____位、尾数位_____位，偏置值Bias = _____。
- IEEE 754规格化数的尾数有隐含的_____前缀（即实际尾数值 = $1 + \text{尾数字段值}/2^{23}$ ）。
- 补码表示的负数进行算术右移时，高位补_____（0或1），以保持符号不变。
- 无符号数的移位称为_____移位，右移时高位补_____。
- 补码加减运算的核心规则是：连同_____一起参加运算，产生的进位_____。
- Booth算法（补码一位乘）中，当末位 Y_0 与附加位 Y_{-1} 为10时，部分积应加_____。
- Booth算法最后一步_____（需要/不需要）进行移位操作。
- 浮点数 $N = S \times r^i$ 中，S称为_____，决定浮点数的_____；i称为_____，决定浮点数的_____。
- 浮点加减运算的四个步骤依次为：对阶 → _____ → 规格化 → _____。
- 对阶的原则是_____向_____看齐，阶小的尾数每右移1位，阶码_____。
- 补码规格化数的判断标准是：符号位与第1数位_____（相同/不同）。即规格化形式为 $0.1xx\dots$ 或 $1.0xx\dots$ 。
- 浮点运算溢出判断：阶码为 $01, xx\dots x$ 表示_____（需中断处理），阶码为 $10, xx\dots x$ 按_____处理。
- 汉明码检测位数k需满足不等式：_____（用n和k表示）。
- 汉明码的检测位应放在_____的位置上（写出位置规律）。

第3章 · RISC-V汇编与指令系统（15题）

- RISC-V架构共有_____个通用寄存器，其中_____号寄存器（x0）恒为0。
- RISC-V的六种指令格式为：R-type、_____、_____、_____、_____和_____。
- 寄存器x1的ABI名称为_____，在过程调用中用于保存_____。
- 寄存器x2的ABI名称为_____，在过程调用中用作_____。
- 函数调用时，前8个参数通过_____寄存器传递，返回值通过_____传回调用者。
- R型指令的三个操作数（两源一目）全部来自_____，这体现了RISC-V的_____架构特征。
- `ld` 指令属于_____型指令，`sd` 指令属于_____型指令。
- I-type指令中包含_____位立即数，使用时会进行_____至32/64位。
- `lui` 指令将20位立即数加载到目标寄存器的_____，低12位补0。
- `jal` 指令属于_____型指令，其跳转范围为_____。
- B型指令（如 `beq`）的立即数为13位，但bit[0]恒为0、隐含×2，因此有效偏移为12位，分支范围为_____。
- 无条件跳转伪指令 `j label` 等效于真实指令_____。
- 函数返回伪指令 `ret` 等效于真实指令_____。
- 叶子过程（不再调用其他函数的过程）可以不使用栈，原因是_____。
- 数组遍历中，_____方式每次迭代比索引方式少一条 `slli` 指令，因此更高效。

第4章 · 处理器设计（13题）

- 冯·诺依曼架构中指令和数据共用_____，在流水线中会导致_____。
- 哈佛架构将_____和_____分离，使取指与访存可以同时进行。
- 单周期处理器数据通路包含的状态单元有：PC（程序计数器）、_____、_____和_____。
- 1-bit ALU 支持的基本运算为：_____、_____和_____。
- 使用1位ALU实现减法（ $a-b$ ）时，需要设置Binvert = _____、CarryIn = _____。

- 32位ALU的Zero标志在所有32位输出 _____ 时置1, 该标志主要用于 _____ 指令的判断。
- ALU控制信号由 _____ 字段和 _____ 字段共同决定。
- 控制信号 _____ 决定ALU的第二操作数来自寄存器 (=0) 还是立即数 (=1)。
- 控制信号 _____ 决定写回寄存器的数据来自ALU结果 (=0) 还是存储器读出数据 (=1)。
- 单周期处理器的时钟周期由 _____ 指令的执行时间决定, 因为该指令的数据通路路径最长。
- `ld` 指令使用的数据通路依次为: PC → _____ → 寄存器堆 → _____ → DM → MUX → 寄存器堆。
- ROM-based控制逻辑中, ROM的地址输入端是 _____, 输出端是 _____。
- `beq` 指令的Branch控制信号应设置为 _____ (0或1)。

第5章 · 流水线处理器 (18题)

- 经典五级流水线的阶段为: IF (_____)、ID (_____)、EX (执行)、MEM (访存)、WB (写回)。
- IF/ID流水线寄存器存储的两项核心内容是: _____ 和 _____。
- ID/EX流水线寄存器需要传递的控制信号包括 (任写4个): _____、_____、MemtoReg、MemRead、MemWrite、Branch、ALUOp等。
- 流水线的理想加速比等于 _____ (各级延迟均衡时)。
- 流水线的理想CPI为 _____, 即每个时钟周期完成一条指令。
- 流水线中的三种冒险 (Hazard) 类型是: _____、_____ 和 _____。
- 解决结构冒险的典型方案是采用 _____ (即分离指令存储器和数据存储器)。
- 解决数据冒险的核心技术是 _____ (英文术语或中文均可)。
- 前推 (Forwarding) 技术中, 旁路数据可能来自 _____ 流水线寄存器或 _____ 流水线寄存器。
- Load-Use数据冒险 _____ (能/不能) 仅通过前推解决, 必须插入 _____ 个stall (气泡)。
- 检测到Load-Use冒险后, Hazard Detection Unit需要做三件事: ① 保持 _____ 寄存器不变 (同一指令重新译码)、② 保持PC不变 (不取新指令)、③ 将ID/EX的 _____ (插入气泡/nop)。
- 1位分支预测器有 _____ 种状态, 2位分支预测器有 _____ 种状态。
- 2位预测器 (饱和计数器) 需要连续 _____ 次预测错误才会改变预测方向, 因此对 _____ 有容忍能力。
- BTB的全称是 _____, 用于缓存分支指令的 _____。
- 控制冒险中, 若分支预测错误, 需要 _____ 已取入流水线的错误路径指令 (将其变为nop)。
- 发生异常时, 硬件自动将异常指令的PC保存到 _____ 寄存器, 异常原因编码保存到 _____ 寄存器。
- 流水线中异常处理的原则是 _____: 即异常指令之前的所有指令正常完成, 异常指令及其后的指令全部被冲刷。
- 将分支决策从MEM阶段提前到ID阶段, 可使分支预测错误时的flush代价从 _____ 条指令减少到 _____ 条。

第6章 · 存储器 (12题)

- 存储器层次结构自上而下依次为: 寄存器 → _____ → _____ → 辅助存储器 (SSD/HDD)。
- 局部性原理包括 _____ 局部性和 _____ 局部性, 两者是Cache能够有效工作的理论基础。
- 时间局部性是指: 刚被访问过的数据项在短期内很可能 _____。
- 空间局部性是指: 某地址附近的数据项在短期内很可能 _____。
- SRAM的存储单元需要 _____ 个晶体管 (双稳态触发器), DRAM的存储单元需要 _____ 个晶体管和 _____ 个电容 (1T1C)。
- DRAM需要定期刷新的原因是 _____, 通常每 _____ 刷新一遍所有行。
- Cache的三种地址映射方式为: 直接映射、_____ 和 _____。
- 三种映射方式中, 冲突缺失 (Conflict Miss) 最严重的是 _____, 完全没有冲突缺失的是 _____。
- Cache替换算法中, 命中率最高的是 _____ (英文缩写), 实际中为降低硬件开销常采用 _____。
- Flash存储器属于 _____ 存储器 (填"易失性"或"非易失性"), 其擦除操作以 _____ 为单位进行。
- Write-through (写直达) 策略下, CPU写数据时需同时更新 _____ 和 _____。
- 主存4GB (32位地址), Cache 64KB, 块大小64B, 采用4路组相联映射时:
① 块内偏移(Offset) = _____ 位 ② 组索引(Index) = _____ 位 ③ 标记(Tag) = _____ 位

第7章 · 系统总线与输入输出 (10题)

- 系统总线按功能分为: _____、_____ 和控制总线。
- 总线带宽公式为: 带宽 = 总线宽度 × _____。
- 三种总线仲裁方式为: _____、集中式并行仲裁和 _____。
- 同步通信使用统一的 _____ 协调收发双方; 异步通信则不依赖统一时钟, 而是通过 _____ 信号来协调。
- 半同步通信在同步通信基础上增加了 _____ 信号, 允许慢速设备插入等待周期。
- 三种I/O控制方式中, CPU效率最低的是 _____, CPU效率最高的是 _____。
- DMA的三种工作方式中, 每次只传送一个字并与CPU交替使用总线的是 _____ 方式。
- 中断处理的基本流程为: 中断请求 → 关中断 → 保存断点 → _____ → 执行ISR → 恢复现场 → _____ → 返回。
- DMA的工作过程为: CPU初始化DMA控制器 → DMA _____ → DMA传送数据 → 传送完成发 _____ 通知CPU。
- 菊花链 (Daisy Chain) 仲裁中, 设备离仲裁器越 _____, 优先级越高。

重点知识点填空 · 答案列表

按题目顺序列出标准答案，分号（；）分隔同一题中的多个空。

第1章 · 计算机系统概论（10题）

1. 控制器；存储器
2. 存储程序
3. 电子管；超大规模集成电路（VLSI）
4. 流水线
5. IC（指令条数）；T（时钟周期）
6. $f / (CPI \times 10^6)$
7. 指令存储器（IM）；数据存储器（DM）
8. ISA（指令集架构）
9. 指令系统；系统结构
10. Cache；改进型哈佛架构

第2章 · 计算机的运算方法（18题）

1. 反码；补码；移码
2. 补码；移码
3. -128；+127
4. [X]₂⁺ 连同符号位在内，每位取反，末位加1
5. 32；1；8；23；127
6. "1."
7. 1
8. 逻辑；0
9. 符号位；自然丢掉
10. [-X]₂⁺
11. 不需要
12. 尾数；精度；阶码；表示范围
13. 尾数求和/差；溢出判断
14. 小阶；大阶；+1
15. 不同
16. 上溢；机器零
17. $2^k \geq n + k + 1$
18. 2^i ($i = 0, 1, 2, \dots$ 即第 1, 2, 4, 8, ... 位)

第3章 · RISC-V汇编与指令系统（15题）

1. 32；x0
2. I-type；S-type；B-type；U-type；J-type
3. ra；返回地址
4. sp；栈指针
5. a0-a7 (x10-x17)；a0-a1
6. 寄存器堆；Load/Store
7. I-type；S-type
8. 12；符号扩展
9. 高20位 [31:12]
10. J-type； ± 1 MB
11. ± 4 KB
12. jal x0, label
13. jalr x0, 0(ra)
14. ra 未被修改，且仅用临时/参数寄存器即可
15. 指针

第4章 · 处理器设计（13题）

1. 同一套存储器；结构冒险
2. 指令存储器（IM）；数据存储器（DM）
3. 指令存储器（IM）；数据存储器（DM）；寄存器堆
4. AND；OR；ADD
5. 1；1

6. 均为0; beq
7. ALUOp; funct3 + funct7
8. ALUSrc
9. MemtoReg
10. Id
11. IM (指令存储器) ; ALU (地址计算)
12. opcode; 各控制信号的值
13. 1

第5章 • 流水线处理器 (18题)

1. 取指; 译码
2. PC+4; 指令字 (32 bit)
3. RegWrite; ALUSrc
4. 流水线级数 k (即 5)
5. 1
6. 结构冒险; 数据冒险; 控制冒险
7. 哈佛架构 (分离 I-Cache / D-Cache)
8. 前推 / 转发 (Forwarding / Bypassing)
9. EX/MEM; MEM/WB
10. 不能; 1
11. IF/ID; 控制信号清零
12. 2; 4
13. 2; 单次波动
14. Branch Target Buffer (分支目标缓冲) ; 目标地址
15. flush (冲刷)
16. SEPC; SCAUSE
17. 精确异常
18. 2; 1

第6章 • 存储器 (12题)

1. Cache; 主存 (DRAM)
2. 时间; 空间
3. 再次被访问
4. 被访问
5. 6; 1; 1
6. 电容漏电; 64 ms
7. 全相联映射; 组相联映射
8. 直接映射; 全相联映射
9. LRU; 伪 LRU
10. 非易失性; 块
11. Cache; 主存
12. 6; 8; 18

第7章 • 系统总线与输入输出 (10题)

1. 地址总线; 数据总线
2. 工作频率
3. 菊花链 (Daisy Chain) ; 分布式仲裁
4. 时钟信号; 握手/应答 (Handshake)
5. 等待 (Wait)
6. 程序查询 (轮询) ; DMA
7. 周期窃取 (Cycle Stealing)
8. 识别中断源; 开中断
9. 请求总线 (HRQ) ; 中断 (EOP)
10. 近

重点知识点填空·题目与答案对照

绿色加粗部分为标准答案，供自检对照。部分题目答案不唯一，合理即得分。

第1章·计算机系统概论（10题）

- 冯·诺依曼架构中，计算机由五大部件组成：运算器、控制器、存储器、输入设备和输出设备。
- 冯·诺依曼架构的核心思想是 存储程序，即程序执行前必须装入存储器。
- 第一代计算机使用 电子管 作为基础器件，第四代使用 超大规模集成电路（VLSI）。
- 计算机体系结构7个伟大思想中，“通过 流水线 提高性能”是流水线处理器设计的基础。
- CPU执行时间 = IC（指令条数） × CPI × T（时钟周期）。
- MIPS（百万指令每秒）= $f / (CPI \times 10^6)$ 。
- 哈佛架构与冯·诺依曼架构的主要区别在于 指令存储器（IM） 和 数据存储器（DM） 是否分离。
- 计算机层次结构中，位于硬件和软件之间的分界面是 ISA（指令集架构）。
- 系列机的核心特征是基本 指令系统 相同、基本 系统结构 相同，以保证软件兼容。
- 现代处理器外部总线为冯·诺依曼结构，但由于内部 Cache 的存在，实际上类似于 改进型哈佛架构。

第2章·计算机的运算方法（18题）

- 机器数的四种编码方式为：原码、反码、补码 和 移码。
- 在四种编码中，+0和-0编码相同（即只有一个0）的是 补码 和 移码。
- 8位补码表示整数的范围是 -128 到 +127。
- 已知[X]_补，求[-X]_补的方法是：[X]_补 连同符号位在内，每位取反，末位加 1。
- IEEE 754单精度浮点数共 32 位，其中符号位 1 位、指数位 8 位、尾数位 23 位，偏置值Bias = 127。
- IEEE 754规格化数的尾数有隐含的“1.”前缀。
- 补码表示的负数进行算术右移时，高位补 1，以保持符号不变。
- 无符号数的移位称为 逻辑 移位，右移时高位补 0。
- 补码加减运算的核心规则是：连同 符号位 一起参加运算，产生的进位 自然丢掉。
- Booth算法（补码一位乘）中，当末位Y_n与附加位Y₋₁为10时，部分积应加 [-X]_补。
- Booth算法最后一步 不需要（需要/不需要）进行移位操作。
- 浮点数 $N = S \times r^i$ 中，S称为 尾数，决定浮点数的 精度；j称为 阶码，决定浮点数的 表示范围。
- 浮点加减运算的四个步骤依次为：对阶 → 尾数求和/差 → 规格化 → 溢出判断。
- 对阶的原则是 小阶向大阶看齐，阶小的尾数每右移1位，阶码 +1。
- 补码规格化数的判断标准是：符号位与第1数位 不同。即规格化形式为 0.1xx... 或 1.0xx...
- 浮点运算溢出判断：阶码为 01,xx...x 表示 上溢（需中断处理），阶码为 10,xx...x 按 机器零 处理。
- 汉明码检测位数k需满足不等式： $2^k \geq n + k + 1$ 。
- 汉明码的检测位应放在 2^i (i = 0, 1, 2, ... 即第 1, 2, 4, 8, ... 位) 的位置上。

第3章·RISC-V汇编与指令系统（15题）

- RISC-V架构共有 32 个通用寄存器，其中 x0 寄存器恒为0。
- RISC-V的六种指令格式为：R-type、I-type、S-type、B-type、U-type 和 J-type。
- 寄存器x1的ABI名称为 ra，在过程调用中用于保存 返回地址。
- 寄存器x2的ABI名称为 sp，在过程调用中用作 栈指针。
- 函数调用时，前8个参数通过 a0-a7 (x10-x17) 寄存器传递，返回值通过 a0-a1 传回调用者。
- R型指令的三个操作数（两源一目）全部来自 寄存器堆，这体现了RISC-V的 Load/Store 架构特征。
- ld 指令属于 I 型指令，sd 指令属于 S 型指令。
- I-type指令中包含 12 位立即数，使用时会进行 符号扩展 至32/64位。
- lui 指令将20位立即数加载到目标寄存器的 高20位 [31:12]，低12位补0。
- jal 指令属于 J 型指令，其跳转范围为 ±1 MB。
- B型指令（如 beq）的立即数为13位，但bit[0]恒为0、隐含×2，因此有效偏移为12位，分支范围为 ±4 KB。
- 无条件跳转伪指令 j label 等效于真实指令 jal x0, label。
- 函数返回伪指令 ret 等效于真实指令 jalr x0, 0(ra)。
- 叶子过程（不再调用其他函数的过程）可以不使用栈，原因是 ra 未被修改，且仅用临时/参数寄存器即可完成。
- 数组遍历中，指针 方式每次迭代比索引方式少一条 slli 指令，因此更高效。

第4章·处理器设计（13题）

- 冯·诺依曼架构中指令和数据共用 同一套存储器，在流水线中会导致 结构冒险。
- 哈佛架构将 指令存储器（IM） 和 数据存储器（DM） 分离，使取指与访存可以同时进行。
- 单周期处理器数据通路包含的状态单元有：PC（程序计数器）、指令存储器（IM）、数据存储器（DM） 和 寄存器堆。
- 1-bit ALU 支持的基本运算为：AND、OR 和 ADD。
- 使用1位ALU实现减法（a-b）时，需要设置Binvert = 1、CarryIn = 1。

- 32位ALU的Zero标志在所有32位输出 均为0 时置1，该标志主要用于 beq 指令的判断。
- ALU控制信号由 ALUOp 字段和 funct3 + funct7 字段共同决定。
- 控制信号 ALUSrc 决定ALU的第二操作数来自寄存器 (=0) 还是立即数 (=1)。
- 控制信号 MemtoReg 决定写回寄存器的数据来自ALU结果 (=0) 还是存储器读出数据 (=1)。
- 单周期处理器的时钟周期由 ld 指令的执行时间决定，因为该指令的数据通路路径最长。
- ld 指令使用的数据通路依次为：PC → IM (指令存储器) → 寄存器堆 → ALU (地址计算) → DM → MUX → 寄存器堆。
- ROM-based控制逻辑中，ROM的地址输入端是 opcode，输出端是 各控制信号的值 (拼接而成)。
- beq 指令的Branch控制信号应设置为 1 (0或1)。

第5章 · 流水线处理器 (18题)

- 经典五级流水线的阶段为：IF (取指)、ID (译码)、EX (执行)、MEM (访存)、WB (写回)。
- IF/ID流水线寄存器存储的两项核心内容是：PC+4 和 指令字 (32 bit)。
- ID/EX流水线寄存器需要传递的控制信号包括 (任写4个)：RegWrite、ALUSrc、MemtoReg、MemRead、MemWrite、Branch、ALUOp等。
- 流水线的理想加速比等于 流水线级数 k (即 5) (各级延迟均衡时)。
- 流水线的理想CPI为 1，即每个时钟周期完成一条指令。
- 流水线中的三种冒险 (Hazard) 类型是：结构冒险、数据冒险 和 控制冒险。
- 解决结构冒险的典型方案是采用 哈佛架构 (分离 I-Cache / D-Cache)。
- 解决数据冒险的核心技术是 前推 / 转发 (Forwarding / Bypassing)。
- 前推 (Forwarding) 技术中，旁路数据可能来自 EX/MEM 流水线寄存器或 MEM/WB 流水线寄存器。
- Load-Use数据冒险 不能 (能/不能) 仅通过前推解决，必须插入 1 个stall (气泡)。
- 检测到Load-Use冒险后，Hazard Detection Unit需要做三件事：① 保持 IF/ID 寄存器不变 (同一指令重新译码)、② 保持PC不变 (不取新指令)、③ 将ID/EX的 控制信号清零 (插入气泡/nop)。
- 1位分支预测器有 2 种状态，2位分支预测器有 4 种状态。
- 2位预测器 (饱和计数器) 需要连续 2 次预测错误才会改变预测方向，因此对 单次波动 有容忍能力。
- BTB的全称是 Branch Target Buffer (分支目标缓冲)，用于缓存分支指令的 目标地址。
- 控制冒险中，若分支预测错误，需要 flush (冲刷) 已取入流水线的错误路径指令 (将其变为nop)。
- 发生异常时，硬件自动将异常指令的PC保存到 SEPC 寄存器，异常原因编码保存到 SCAUSE 寄存器。
- 流水线中异常处理的原则是 精确异常：即异常指令之前的所有指令正常完成，异常指令及其后的指令全部被冲刷。
- 将分支决策从MEM阶段提前到ID阶段，可使分支预测错误时的flush代价从 2 条指令减少到 1 条。

第6章 · 存储器 (12题)

- 存储器层次结构自上而下依次为：寄存器 → Cache → 主存 (DRAM) → 辅助存储器 (SSD/HDD)。
- 局部性原理包括 时间 局部性和 空间 局部性，两者是Cache能够有效工作的理论基础。
- 时间局部性是指：刚被访问过的数据项在短期内很可能 再次被访问。
- 空间局部性是指：某地址附近的数据项在短期内很可能 被访问。
- SRAM的存储单元需要 6 个晶体管 (双稳态触发器)，DRAM的存储单元需要 1 个晶体管和 1 个电容 (1T1C)。
- DRAM需要定期刷新的原因是 电容存在漏电流，电荷逐渐流失，通常每 64 ms 刷新一遍所有行。
- Cache的三种地址映射方式为：直接映射、全相联映射 和 组相联映射。
- 三种映射方式中，冲突缺失 (Conflict Miss) 最严重的是 直接映射，完全没有冲突缺失的是 全相联映射。
- Cache替换算法中，命中率最高的是 LRU (英文缩写)，实际中为降低硬件开销常采用 伪LRU。
- Flash存储器属于 非易失性 存储器 (填“易失性”或“非易失性”)，其擦除操作以 块 为单位进行。
- Write-through (写直达) 策略下，CPU写数据时需同时更新 Cache 和 主存。
- 主存4GB (32位地址)，Cache 64KB，块大小64B，采用4路组相联映射时：
① 块内偏移(Offset) = 6 位 ② 组索引(Index) = 8 位 ③ 标记(Tag) = 18 位

第7章 · 系统总线与输入输出 (10题)

- 系统总线按功能分为：地址总线、数据总线 和控制总线。
- 总线带宽公式为：带宽 = 总线宽度 × 工作频率。
- 三种总线仲裁方式为：菊花链 (Daisy Chain)、集中式并行仲裁和 分布式仲裁。
- 同步通信使用统一的 时钟信号 协调收发双方；异步通信则不依赖统一时钟，而是通过 握手/应答 (Handshake) 信号来协调。
- 半同步通信在同步通信基础上增加了 等待 (Wait) 信号，允许慢速设备插入等待周期。
- 三种I/O控制方式中，CPU效率最低的是 程序查询 (轮询)，CPU效率最高的是 DMA。
- DMA的三种工作方式中，每次只传送一个字并与CPU交替使用总线的是 周期窃取 (Cycle Stealing) 方式。
- 中断处理的基本流程为：中断请求 → 关中断 → 保存断点 → 识别中断源 → 执行ISR → 恢复现场 → 开中断 → 返回。
- DMA的工作过程为：CPU初始化DMA控制器 → DMA 请求总线 (HRQ) → DMA传送数据 → 传送完成发 中断 (EOP) 通知CPU。
- 菊花链 (Daisy Chain) 仲裁中，设备离仲裁器越 近，优先级越高。

各章节课后练习复习题

共52道练习题，覆盖全部五章。题目包含简答、分析、计算、设计等类型，建议全部独立完成后对照答案。

第1章 · 计算机系统概论

1. 简述冯·诺依曼架构计算机的六大特征。
2. 比较冯·诺依曼架构与哈佛架构的异同，并说明现代处理器为何采用改进型哈佛架构。
3. 某程序包含指令100万条，CPI=1.5，CPU主频=2 GHz。求：
(1) CPU执行时间（秒）。
(2) 若优化后CPI降为1.2，其他不变，求加速比。
4. 计算机体系结构中有哪7个伟大思想？请分别简要说明。
5. 某CPU主频3 GHz，执行某程序耗时2 ms，程序指令总数为 6×10^6 条。求该程序的平均CPI和CPU的MIPS值。

第2章 · 计算机的运算方法

6. 已知 $X = -13$ ， $Y = +7$ ，字长8位（含1位符号位）：
(1) 写出 X 和 Y 的原码、反码、补码。
(2) 用补码计算 $X+Y$ 和 $X-Y$ ，并判断是否溢出。
7. 将十进制数 -320 转换为 IEEE 754 单精度浮点数的十六进制表示。
8. 已知 $X = -5/256 = -0.00000101_2 \times 2^0$ ， $Y = -59/1024$ ，使用浮点数补码计算 $X-Y$ 。阶符取2位，阶码取3位，尾数取9位（忽略对阶及尾数舍入）。
9. 用Booth算法计算 $X \times Y$ 。已知 $X = +3$ （被乘数）， $Y = -5$ （乘数），字长5位（含符号位）。写出完整计算过程。
10. 数据 0101 按偶校验原则配置汉明码，写出完整过程。如果收到的汉明码为 0110101，判断是否有错？若有错，指出错误位并纠正。
11. 16位浮点数（阶符1位+阶码4位+数符1位+尾数10位），基数 $r=2$ 。求最大正数、最小正数、最大负数、最小负数的真值（用2的幂表示即可）。
12. 已知 $X = 0.1101 \times 2^{01}$ ， $Y = (-0.1010) \times 2^{11}$ ，用补码计算 $X+Y$ （阶符2位，阶码3位，尾数6位），写出对阶、求和、规格化全过程。

第3章 RISC-V汇编及其指令系统

1. 简述RISC-V架构的主要设计特点（至少列出4点）。

2. 写出以下指令的格式类型及其功能: (a) add x5,x6,x7 (b) sub x5,x6,x7 (c) ld x8,16(x9) (d) sd x10,32(x11) (e) beq x12,x13,label

3. 已知x6=0x1000, x7=0x2000, x8=0x3000, 计算: (a) add x5,x6,x7 $\rightarrow$ x5=? (b) sub x5,x7,x6 $\rightarrow$ x5=? (c) and x5,x6,x8 $\rightarrow$ x5=? (d) or x5,x6,x7 $\rightarrow$ x5=?

4. 将以下C语句翻译为RISC-V汇编: (a) $f = g + h + i - 5$ (b) $A[10] = B[5] + c$ (c) if (i == j) $f = g + h$; else $f = g - h$;

5. 什么是叶子过程? 为什么叶子过程可以不使用栈?

6. 解释为什么B型和J型指令的立即数域要隐含左移1位 ($\times 2$)。

7. 写出swap过程的完整RISC-V汇编 (交换v[k]和v[k+1], 数组基址在a0, k在a1)。

8. 分析数组访问中"索引方式"(A[i])与"指针方式"(*p++)的效率差异, 详细说明原因。

9. 为什么RISC-V中S-type和B-type指令的立即数字段被拆分成多个不连续的段?

10. 写出将一个32位立即数0x12345678加载到x5的完整RISC-V指令序列。

第4章 处理器设计

1. 比较冯·诺依曼架构与哈佛架构, 从5个维度对比 (存储器、总线、并行性、复杂度、应用)。

2. 画出1-bit ALU的逻辑结构 (支持AND/OR/ADD三种运算)。

3. 说明32位ALU中SLT操作的实现原理——为什么需要特殊的处理?

4. 详细描述单周期处理器执行ld x1,40(x2)时各组件的工作流程和数据流向。

5. 给出R-type/sub、ld、sd、beq、jal五类指令的完整控制信号表 (7个信号+ALUOp)。

6. 什么是关键路径? 为什么单周期处理器的时钟周期由ld指令决定? 画出ld的关键路径。

7. 如果要在单周期数据通路中添加一条jm(rd, rs1)指令 (功能: $rd = PC + 4$; $PC = rs1$), 需要修改哪些组件和控制信号?

8. 简述ROM-based控制逻辑的原理, 画出输入输出关系。

9. 解释为什么控制信号在时钟的不同阶段起作用？哪些是组合逻辑输出？哪些在边沿起作用？

10. 单周期处理器执行beq指令时，分支目标地址在哪个组件计算？Zero信号来自哪里？

第5章 流水线处理器

1. 画出经典五级流水线的结构框图，标注四组流水线寄存器及其传递内容。

2. 某处理器:IF=250ps, ID=200ps, EX=300ps, MEM=350ps, WB=150ps, 流水线寄存器=25ps。(a)单周期时钟周期=? (b)流水线时钟周期=? (c)理想加速比 (d)10^9条指令执行时间(无冒险)。

3. 某程序:ld占25%且其中80%导致Load-Use停顿1周期，分支占15%且10%预测错误(惩罚2周期)，无其他冒险。求实际CPI和相对理想CPI的减速比。

4. 分析以下指令序列中的所有数据依赖，指出哪些可通过转发解决: I1:add x1,x2,x3; I2:sub x4,x1,x5; I3:and x6,x1,x7; I4:or x8,x4,x9; I5:add x10,x8,x11

5. 详细解释Load-Use数据冒险:为什么它无法仅通过转发解决? Hazard Detection Unit检测到后如何操作? 画出时空图。

6. 比较1位和2位分支预测器，画出状态转换图。2位预测器在何种程序模式下明显优于1位？

7. 为什么在流水线中，通常将分支判断提前到ID阶段？这样做的代价是什么？

8. 流水线中异常处理为什么要按照指令顺序而不是异常检测顺序来响应？

9. SEPC和SCAUSE在流水线中分别在哪一级被写入？各存什么内容？

10. 为什么流水线不能无限加深（如100级流水线）？列出限制因素。

11. 设计一个能检测所有数据冒险的流水线冒险检测单元，写出其关键判断条件。

12. 某程序:20%ld(5%Load-Use→1 stall),10%分支(50%预测错误→2 flush),5%jal(无条件1 flush)。求CPI。

第6章 存储器

1. 从存储介质、存取方式、在计算机中作用、可保存性四个维度对存储器进行分类，每类举例说明。

2. 画出存储器的层次结构图（金字塔），标注各层的容量和延迟范围。解释为什么层次结构有效。

3. 从存储单元结构、速度、集成度、功耗、是否需要刷新五个维度详细对比SRAM和DRAM。

4. DRAM为什么需要刷新？电容漏电的物理原理是什么？描述三种刷新方式的优缺点。

5. 比较ROM系列（MROM/PROM/EPROM/EEPROM/Flash）在编程方式、擦除方式和可重写次数上的差异。

6. Cache三种地址映射方式（直接/全相联/k路组相联）的优缺点和应用场景。

7. 主存4GB，Cache 128KB，块64B。(a)直接映射 Tag/Index/Offset=? (b)4路组相联 (c)全相联 (d)16路组相联。

8. 比较LRU、FIFO、RAND三种替换算法。为什么实际Cache中常用伪LRU而非真LRU？

9. Write-through和Write-back的优缺点对比。Write-allocate和No-write-allocate分别适合哪种写策略？

10. 计算题:主存访问时间50ns，Cache命中率95%，Cache访问时间5ns。求平均访问时间。若命中率提高到98%，平均访问时间改善多少？

第7章 系统总线与输入输出

1. 系统总线由哪三部分组成？各自是单向还是双向？数据总线宽度对系统有什么影响？

2. 比较三种总线仲裁方式（菊花链、集中式并行、分布式），从优先级灵活性、可靠性、硬件开销三方面分析。

3. 比较同步通信和异步通信。半同步通信如何融合两者优点？分离式通信的优势是什么？

4. 总线64位@100MHz(a)带宽(MB/s)(b)若支持双倍速率(DDR,每时钟2次)带宽? (c)若还支持4突发传输(1地址+4数据)带宽?

5. 从数据传送单位、CPU利用率、硬件开销和适用场景对比程序查询、程序中断和DMA三种I/O方式。

6. 详细描述中断处理的完整流程，说明"关中断"和"开中断"在什么阶段进行及其原因。

7. 比较DMA三种工作方式（突发/周期窃取/透明）的总线利用率和CPU影响。

8. 为什么DMA比中断更适合高速数据传输？（如磁盘→内存，10MB/s以上）

9. 什么是中断嵌套？实现中断嵌套需要哪些条件？

10. 解释为什么现代计算机中使用总线桥(Bridge)结构的多总线体系。画出CPU-主存总线到PCIe到USB的层次结构。

第四部分·模拟试题（三份）

模拟试题（一）

总分：100分 | 时间：120分钟 | 题型：填空+选择+简答+分析+设计

一、填空题（每空1分，共20分）

1. RISC-V中x0寄存器的值恒为_____。
2. R-type指令的6个字段分别是：funct7, rs2, rs1, funct3, rd, _____。
3. jalr指令属于_____型指令。
4. 哈佛架构的最大特点是_____和_____分开存储。
5. 1位分支预测器有_____个状态。
6. 流水线的5个阶段中，EX表示_____阶段。
7. Load-Use冒险需要插入_____个nop。
8. DRAM的每个存储单元由_____个晶体管和1个电容组成。
9. Cache与主存之间的地址映射方式有_____、_____和_____三种。
10. 局部性原理包括_____局部性和_____局部性。
11. 总线带宽 = 总线宽度 × _____。
12. DMA的三种工作方式是_____、周期窃取和透明方式。
13. 发生异常时，SEPC保存_____。
14. RISC-V是典型的_____（Load/Store）架构。
15. slt指令的功能是_____。
16. lui指令将立即数加载到寄存器的_____位。

二、选择题（每题2分，共20分）

1. 下列哪条指令是S-type指令？（ ）
A. add x5,x6,x7 B. ld x5,0(x6) C. sd x5,0(x6) D. beq x5,x6,label
2. RISC-V共有多少个通用寄存器？（ ）
A. 8 B. 16 C. 32 D. 64
3. 以下哪种冒险可以通过前推(Forwarding)解决？（ ）
A. 结构冒险 B. Load-Use数据冒险 C. EX数据冒险 D. 控制冒险
4. 2位分支预测器需要连续几次预测错误才会改变预测方向？（ ）
A. 1次 B. 2次 C. 3次 D. 4次
5. 下列哪种存储器是易失性的？（ ）
A. Flash B. EPROM C. SRAM D. EEPROM
6. Cache替换算法中命中率通常最高的是？（ ）
A. RAND B. FIFO C. LRU D. LFU
7. 下列哪种I/O方式CPU效率最高？（ ）
A. 程序查询 B. 程序中断 C. DMA D. 三者相同
8. 冯诺依曼架构中，指令和数据存储在哪里？（ ）
A. 不同存储器 B. 同一存储器 C. 寄存器中 D. Cache中
9. ALU控制信号ALUOp的位宽是多少？（ ）
A. 1位 B. 2位 C. 3位 D. 4位
10. 流水线各阶段中最慢阶段决定了什么？（ ）
A. 加速比 B. 流水线级数 C. 时钟周期 D. CPI

三、简答题（每题5分，共20分）

1. 简述RISC-V的6种指令格式，并各举一例指令。
2. 说明数据冒险的三种类型，以及各自的解决方式。
3. 比较SRAM和DRAM的主要区别。
4. 简述DMA方式的数据传送过程。

四、分析题（每题10分，共20分）

1. 分析以下指令序列中的数据依赖关系，指出哪些可以通过前推解决，哪些需要停顿：
lw x1, 0(x2)
add x3, x1, x4
sub x5, x6, x7
or x8, x5, x9
2. 某计算机Cache容量为32KB，块大小为32B，采用4路组相联映射。主存地址为32位。求Tag、Index、Block Offset各占多少位？

五、设计题（每题20分，共20分）

1. 将以下C代码翻译为RISC-V汇编语言（要求使用栈保存必要的寄存器）：

```
int sum_array(int arr[], int n) {  
    int sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum += arr[i];  
    }  
    return sum;  
}
```

模拟试题（二）

总分：100分 | 时间：120分钟 | 题型：填空+选择+判断+简答+计算

一、填空题（每空1分，共20分）

- RISC-V的寄存器x1的ABI名为 _____，x2的ABI名为 _____。
- RISC-V指令格式中，I-type的立即数为 _____ 位。
- sd指令的寻址方式是 _____ 寻址。
- 单周期处理器的时钟周期由 _____ 指令的执行时间决定。
- 32位ALU的Zero标志在结果为 _____ 时置1。
- 流水线中，ID/EX寄存器在 _____ 和 _____ 阶段之间。
- 2位分支预测器包含4个状态：强不跳、_____、_____、强跳。
- DRAM需要定期刷新的原因是 _____。
- 半导体存储器按存取方式可分为 _____ 和 _____。
- Cache的块替换算法中，_____ 是最常用的近似LRU的算法。
- 菊花链仲裁方式的特点是优先级由 _____ 决定。
- 总线通信方式中，_____ 通信需要统一的时钟信号。
- 程序中断方式的优点是CPU与 _____ 可并行工作。
- 在流水线中保存异常原因的寄存器是 _____。
- 前推技术中，数据可能来自 _____ 寄存器或 _____ 寄存器。
- 汇编伪指令 _____ 不是真正的机器指令。

二、选择题（每题2分，共20分）

- 下列寄存器中哪个用于保存函数返回值？（ ）
A. x0 B. x1 C. x2 D. x10
- beq指令属于哪种指令格式？（ ）
A. R-type B. I-type C. S-type D. B-type
- 流水线加速比的上限受什么因素限制？（ ）
A. 时钟频率 B. 流水线级数 C. 指令数量 D. 寄存器数量
- 下列哪项不是流水线冒险？（ ）
A. 结构冒险 B. 数据冒险 C. 地址冒险 D. 控制冒险
- 一个DRAM芯片的存储容量为1G×8，其地址线至少需要多少条？（ ）
A. 8 B. 20 C. 30 D. 32
- 属于非易失性存储器的是（ ）
A. SRAM B. DRAM C. Flash D. 寄存器
- 总线仲裁中，哪种方式的可靠性最高？（ ）
A. 菊花链 B. 集中式并行 C. 分布式 D. 三者相同
- DMA在哪种工作方式下与CPU交替使用总线？（ ）
A. 突发方式 B. 周期窃取 C. 透明方式 D. 中断方式
- 控制信号RegWrite=1, MemRead=0, MemWrite=0, ALUSrc=0 对应什么指令？（ ）
A. ld B. sd C. add D. beq
- 采用4路组相联映射的Cache，每组包含几个Cache行？（ ）
A. 1 B. 2 C. 4 D. 8

三、判断题（每题1分，共10分，正确打√，错误打×）

- () RISC-V中x0寄存器可以写入任意值。
- () R-type指令的三个操作数全部来自寄存器。
- () 流水线可以提高单条指令的执行速度。
- () 转发技术可以解决所有类型的数据冒险。
- () Cache利用的是时间局部性和空间局部性原理。
- () SRAM比DRAM的集成度更高。
- () DMA方式下，CPU完全不参与数据传输过程。
- () 同步通信需要握手信号，异步通信需要统一时钟。
- () 2位分支预测器比1位预测器需要更多的状态位。
- () 单周期处理器中，所有指令的执行时间相同。

四、简答题（每题5分，共20分）

- 简述RISC-V中的过程调用约定，包括参数传递、返回值和栈的使用。
- 什么是关键路径？以ld指令为例，画出其在单周期处理器中的关键路径。
- 比较三种I/O控制方式（轮询、中断、DMA）的异同。
- 说明Cache中写策略（Write-through和Write-back）的区别。

五、计算题（每题10分，共30分）

- 某5级流水线处理器，各阶段延迟为：IF=250ps, ID=200ps, EX=300ps, MEM=350ps, WB=150ps，流水线寄存器延迟25ps。
(a) 求流水线时钟周期；(b) 若程序有 10^9 条指令，无冒险，求总执行时间；(c) 求理想加速比。
- 主存容量为1GB，按字节编址。Cache容量为128KB，块大小为128B，采用直接映射。
(a) 主存地址总位数；(b) Cache共有多少行；(c) 地址中Tag、Index、Block Offset各占多少位。
- 某总线宽度为32位，工作频率为66MHz，支持突发传输（一个地址后跟4个数据）。求总线最大数据传输率（MB/s）。

模拟试题（三）

总分：100分 | 时间：120分钟 | 题型：填空+选择+简答+综合

一、填空题 (每空1分, 共20分)

1. 在RISC-V中, 保存寄存器的ABI名为s0-s11, 临时寄存器的ABI名为 _____ 。
2. RISC-V指令中, B-type的立即数字段是 _____ 位, 但有效范围是 $\pm 4\text{KB}$ (因为隐含 _____ 1位) 。
3. auipc指令的功能是 _____ 。
4. 寄存器堆(Register File)有 _____ 个读端口和 _____ 个写端口。
5. 流水线中, 数据冒险的三种类型是RAW、 _____ 和 _____ 。
6. 前推(Forwarding)的别名是 _____ (英文) 。
7. 在流水线中检测Load-Use冒险的单元是 _____ 单元。
8. 存储器层次结构中, _____ 位于寄存器和主存之间。
9. 半导体ROM的五种类型: MROM、PROM、 _____ 、 _____ 、Flash。
10. 总线的 _____ 特性描述了总线一次能传输的数据位数。
11. 菊花链仲裁方式中, 离仲裁器越 _____ 的设备优先级越高。
12. DMA与CPU争用总线时, _____ 方式的DMA仅在不影响CPU时传送数据。
13. 中断处理过程中, 保存和恢复现场的目的是 _____ 。
14. RISC-V中伪指令 _____ 用于加载32位立即数 (由lui+addi组成) 。
15. 单周期处理器的控制信号通常由 _____ (组合逻辑/ROM/微程序) 方式生成。
16. 流水线中, IF/ID寄存器存储的两项主要内容是 _____ 和 _____ 。

二、选择题 (每题2分, 共20分)

1. 以下哪条指令不是I-type? ()
A. addi B. ld C. jalr D. jal
2. 在RISC-V汇编中, 调用函数时返回地址保存在 ()
A. x0 B. x1 C. x2 D. x5
3. 如果流水线有5级, 理想情况下的最大加速比是 ()
A. 1 B. 2.5 C. 5 D. 10
4. 以下哪种情况必须插入流水线气泡? ()
A. EX数据冒险 B. MEM数据冒险 C. Load-Use数据冒险 D. 结构冒险
5. EPROM的擦除方式是 ()
A. 电擦除 B. 紫外线擦除 C. 不可擦除 D. 红外线擦除
6. 下列哪种映射方式下, 冲突缺失(conflict miss)最严重? ()
A. 直接映射 B. 全相联映射 C. 4路组相联 D. 8路组相联
7. 异步通信相对于同步通信的优点是 ()
A. 速度快 B. 简单 C. 兼容不同速度设备 D. 成本低
8. 关于I/O中断, 以下说法正确的是 ()
A. CPU主动查询外设状态 B. 外设主动通知CPU C. 不需要CPU参与 D. 不保存断点
9. 以下哪种情况会导致结构冒险? ()
A. 两条指令访问同一寄存器 B. 单存储器同时取指和读数据 C. 分支指令改变PC D. 前一条指令的结果未就绪
10. 在32位RISC-V中, lui x5, 0x12345指令执行后x5的值是 ()
A. 0x00012345 B. 0x12345000 C. 0x12345 D. 0x54321000

三、简答题 (每题5分, 共20分)

1. 画出单周期处理器的完整数据通路框图 (需标注各组件和关键控制信号) 。
2. 说明流水线中结构冒险、数据冒险、控制冒险各自产生的原因和解决方法。
3. 什么是Cache的命中率? 影响命中率的因素有哪些?
4. 简述中断优先级的判断方式和中断嵌套的实现条件。

四、综合题 (每题10/15/15分, 共40分)

1. (10分) 将以下C函数翻译为RISC-V汇编:

```
int power(int base, int exp) {  
    int result = 1;  
    while (exp > 0) {  
        result = result * base;  
        exp--;  
    }  
    return result;  
}
```

注: RISC-V标准指令集无乘法指令, 用mul伪指令表示即可 (实际由编译器展开) 。

2. (15分) 某5级流水线处理器, 考虑以下指令序列:

```
I1: add x1, x2, x3  
I2: ld x4, 0(x1)  
I3: sub x5, x4, x6  
I4: sd x5, 8(x7)  
I5: beq x5, x0, target
```

(a) 识别所有数据依赖关系; (b) 指出哪些可通过转发解决, 哪些需要停顿; (c) 画出带转发和停顿的流水线时空图。

3. (15分) 设计一个Cache系统: 主存4GB, Cache 256KB, 块大小64B, 采用8路组相联映射, 使用LRU替换。(a) 计算Tag、Index、Block Offset位数; (b) 说明LRU替换策略在此配置下的实现开销; (c) 分析为什么8路组相联在性能上通常是直接映射和全相联的良好折中。

一、填空题答案 (80空)

第1章 • 计算机系统概论 习题答案

1. 简述冯·诺依曼架构六大特征

- ① 计算机由五大部件组成 (运算器、控制器、存储器、输入设备、输出设备)
- ② 指令和数据以同等地位存于存储器, 可按地址寻访
- ③ 指令和数据都用二进制表示
- ④ 指令由操作码和地址码组成
- ⑤ 存储程序: 程序执行前必须装入存储器
- ⑥ 以运算器为中心, 指令通常顺序执行

2. 冯·诺依曼 vs 哈佛架构

相同点: 都是计算机体系结构, 都有CPU+存储器+I/O的基本组成。

不同点: 冯·诺依曼指令和数据共用同一存储器 (单总线), 取指和访存不能同时进行; 哈佛架构指令存储器和数据存储器分开 (双总线), 可取指和访存同时进行。

改进型哈佛: 现代处理器外部总线为冯·诺依曼, 但内部通过I-Cache和D-Cache实现数据和指令的分离访问, 兼具两者的优点。

3. CPI/执行时间计算

$$(1) T = 1/f = 1/(2 \times 10^9) = 0.5 \text{ ns}$$

$$\text{执行时间} = IC \times CPI \times T = 10^6 \times 1.5 \times 0.5 \times 10^{-9} = 0.75 \text{ ms}$$

$$(2) \text{优化后执行时间} = 10^6 \times 1.2 \times 0.5 \times 10^{-9} = 0.6 \text{ ms}$$

$$\text{加速比} = 0.75/0.6 = 1.25$$

4. 7个伟大思想

- ① 使用抽象简化设计
- ② 加速经常性事件
- ③ 通过并行提高性能
- ④ 通过流水线提高性能
- ⑤ 通过预测提高性能
- ⑥ 存储层次
- ⑦ 通过冗余提高可靠性

5. MIPS计算

$$T = 1/(3 \times 10^9) = 0.333 \text{ ns}$$

$$CPI = \text{执行时间} / (IC \times T) = 2 \times 10^{-3} / (6 \times 10^6 \times 0.333 \times 10^{-9}) = 1.0$$

$$MIPS = f / (CPI \times 10^6) = 3 \times 10^9 / (1.0 \times 10^6) = 3000$$

第2章 • 计算机的运算方法 习题答案

6. X=-13, Y=+7 补码运算

$$(1) X = -13: [X]_0 = 1,0001101, [X]_r = 1,1110010, [X]_s = 1,1110011$$

$$Y = +7: [Y]_0 = [Y]_r = [Y]_s = 0,0000111$$

$$(2) [X+Y]_s = [X]_s + [Y]_s = 1,1110011 + 0,0000111 = 1,1111010 \rightarrow \text{真值} = -6 \checkmark$$

$$[X-Y]_s = [X]_s + [-Y]_s = 1,1110011 + 1,1111001 = 1,1101100 \rightarrow \text{真值} = -20 \checkmark \text{ (无溢出, 因双符号位一致)}$$

7. -320 的 IEEE 754 单精度表示

$$320 = 256 + 64 = 2^8 + 2^6 = 101000000_2 = 1.01000000 \times 2^8$$

$$s = 1 \text{ (负数)}$$

$$E = 8 + 127 = 135 = 10000111_2$$

$$M = 010000000000000000000000$$

$$\text{结果: } 1 \ 10000111 \ 010000000000000000000000 \rightarrow \mathbf{0xC3A00000}$$

8. 浮点数 X-Y 补码计算

$$X = -5/256 = -0.0000101_2 = -0.101 \times 2^{-5}$$

$$Y = -59/1024 \approx -0.0000111011_2 \approx -0.111011 \times 2^{-4}$$

$$[X]_s = 11,011; 11.011000$$

$$[Y]_s = 11,100; 11.000101$$

- ① 对阶: $\Delta_j = -1$, x向y看齐 $\rightarrow [x]_+ = 11,100; 11.101100$
- ② 尾数求差: $[Sx]_+ + [-Sy]_+ = 11.101100 + 00.111011 = 00.100111$
- ③ 结果: $[X-Y]_+ = 11,100; 00.100111$ (已规格化)

9. Booth算法 $X=+3, Y=-5$

$[X]_+ = 0,0011, [-X]_+ = 1,1101, [Y]_+ = 1,1011$

部分积 / 乘数 / 附加位 $\rightarrow$ 操作

0,0000 1,1011 0 $\rightarrow Y_0Y_{-1}=10$, 加 $[-X]_+ = 1,1101 \rightarrow 1,1101 \rightarrow$ 右移 $\rightarrow 1,1110 1,1101 1$

$Y_0Y_{-1}=11$, 右移 $\rightarrow 1,1111 0,1110 1$

$Y_0Y_{-1}=01$, 加 $[X]_+ = 0,0011 \rightarrow 0,0010 \rightarrow$ 右移 $\rightarrow 0,0001 0,0111 0$

$Y_0Y_{-1}=10$, 加 $[-X]_+ = 1,1110 \rightarrow$ 右移 $\rightarrow 1,1111 0,0011 1$

$Y_0Y_{-1}=11$, 不右移 (最后一步) $\rightarrow [X \times Y]_+ = 1,11110001 \rightarrow$ 真值 $= -15 \checkmark$

10. 汉明码 0101 偶校验

$n=4, 2^k \geq n+k+1 \rightarrow k=3$, 汉明码共7位, 检测位在1,2,4位:

位序 1 2 3 4 5 6 7 $\rightarrow C_1 C_2 0 C_4 1 0 1$

$C_1 = \text{位}3 \oplus \text{位}5 \oplus \text{位}7 = 0 \oplus 1 \oplus 1 = 0$

$C_2 = \text{位}3 \oplus \text{位}6 \oplus \text{位}7 = 0 \oplus 0 \oplus 1 = 1$

$C_4 = \text{位}5 \oplus \text{位}6 \oplus \text{位}7 = 1 \oplus 0 \oplus 1 = 0$

$\therefore$ 汉明码 = **0100101**

若收到 0110101: $P_1=0 \oplus 0 \oplus 1 \oplus 1=0, P_2=1 \oplus 0 \oplus 0 \oplus 1=0, P_4=0 \oplus 1 \oplus 0 \oplus 1=0 \rightarrow$ 无错

11. 16位浮点数范围

阶码4位(补码), 尾数10位(补码规格化):

最大正数 $= 2^7 \times (1-2^{-10})$ (阶码最大+7)

最小正数 $= 2^{-8} \times 2^{-1}$ (阶码最小-8, 尾数最小)

最大负数 $= -2^{-8} \times 2^{-1}$

最小负数 $= -2^7 \times 1$

12. 浮点 $X+Y$ 全过程

$[x]_+ = 00,01; 00.1101, [y]_+ = 00,11; 11.0110$

① 对阶: $\Delta_j = [jx]_+ - [jy]_+ = 00,01 - 00,11 = 11,10(-2)$, x向y看齐

$[x]_+ = 00,11; 00.001101 \rightarrow$ 舍入得 00.0011

② 尾数和: $[Sx]_+ + [Sy]_+ = 00.0011 + 11.0110 = 11.1001$

③ $[x+y]_+ = 00,11; 11.1001$, 左规: $00,10; 11.0010$

$\therefore x+y = (-0.1110) \times 2^{10} = -0.1110 \times 2^2$

第3章 RISC-V汇编 (第1-16题)

1. 32; 0
2. I-type; S-type; J-type
3. ra; 返回地址
4. sp; 栈指针
5. a0-a7(x10-x17); a0-a1
6. 寄存器; 寄存器
7. I-type; S-type
8. 12; 符号扩展
9. 高20位 ([31:12])
10. J-type; $\pm 1MB$
11. $\pm 4KB$
12. jal x0, label
13. jalr x0, 0(ra)
14. ra寄存器未被修改 (叶子过程不调用其他函数)
15. 指针方式

第4章 处理器设计 (第17-29题)

1. 同一; 结构冒险
2. 指令; 数据
3. 寄存器堆(Register File); 流水线寄存器
4. AND; ADD

5. 1; 1
6. 均为0; beq
7. ALUOp; funct3+funct7
8. ALUSrc
9. MemtoReg
10. ld
11. IM; ALU (地址计算)
12. opcode; 各控制信号值
13. 1

第5章 流水线 (第30-49题)

1. 取指; 译码
2. PC+4; 指令字 (32bit)
3. RegWrite; ALUSrc (另可填MemtoReg/MemRead/MemWrite/Branch/ALUOp)
4. 流水线级数 (即5)
5. 1
6. 结构冒险; 数据冒险; 控制冒险
7. 哈佛架构 (分离IM/DM)
8. 前推/转发(Forwarding/Bypassing)
9. EX/MEM; MEM/WB
10. 不能; 1
11. IF/ID寄存器; 控制信号清零(插入nop)
12. 2; 4
13. 2; 单次波动
14. Branch Target Buffer (分支目标缓冲); 目标地址
15. flush (冲刷)
16. SEPC; SCAUSE
17. 精确异常
18. 2; 1

第6章 存储器 (第50-63题)

1. Cache; 主存 (DRAM)
2. 时间; 空间
3. 再次被访问
4. 被访问
5. 6; 1; 1
6. 电容漏电; 64ms
7. 全相联映射; 组相联映射
8. 直接映射; 全相联映射
9. LRU; 伪LRU
10. 非易失性; 块
11. Cache和主存; Cache
12. 6; 8; 18

第7章 系统总线与I/O (第64-73题)

1. 地址总线; 数据总线
2. 工作频率
3. 菊花链(Daisy Chain); 分布式
4. 时钟信号; 握手应答
5. 等待(Wait)
6. DMA; 程序查询 (轮询)
7. 周期窃取(Cycle Stealing)
8. 识别中断源; 开中断
9. 请求总线; 中断
10. 近

二、章节练习题参考答案要点

第3章 RISC-V汇编及其指令系统

▶ 1. 参考答案

▶ 2. 参考答案

▶ 3. 参考答案

▶ 4. 参考答案

▶ 5. 参考答案

▶ 6. 参考答案

▶ 7. 参考答案

▶ 8. 参考答案

▶ 9. 参考答案

▶ 10. 参考答案

第4章 处理器设计

▶ 1. 参考答案

▶ 2. 参考答案

▶ 3. 参考答案